

# Kötü Projede Agent Tahmin Eder: Yazılımda Niyet ve İz

Ömer Faruk Yavuz

Mayıs 2026

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Giriş</b>                                       | <b>6</b>  |
| <b>2</b> | <b>Teknik Borç Takip Listesi</b>                   | <b>7</b>  |
| 2.1      | 1. Kod Geometrisi Borcu . . . . .                  | 7         |
| 2.2      | 2. Davranış Geometrisi Borcu . . . . .             | 7         |
| 2.3      | 3. Spec Borcu . . . . .                            | 7         |
| 2.4      | 4. Test Borcu . . . . .                            | 8         |
| 2.5      | 5. Developer Experience Borcu . . . . .            | 8         |
| 2.6      | 6. Agent Experience Borcu . . . . .                | 9         |
| 2.7      | 7. Dokümantasyon Borcu . . . . .                   | 9         |
| 2.8      | 8. Domain ve Anlam Borcu . . . . .                 | 9         |
| 2.9      | 9. Mimari Borç . . . . .                           | 10        |
| 2.10     | 10. Güvenlik ve Privacy Borcu . . . . .            | 10        |
| 2.11     | 11. Operasyonel Borç . . . . .                     | 10        |
| 2.12     | 12. Feedback Loop Borcu . . . . .                  | 11        |
| 2.13     | Teknik Borç Kaydı İçin Önerilen Format . . . . .   | 11        |
| 2.14     | Örnek Teknik Borç Kaydı . . . . .                  | 11        |
| <b>3</b> | <b>Dil Kullanımı ve Profesyonel Güvenilirlik</b>   | <b>13</b> |
| 3.1      | Giriş . . . . .                                    | 13        |
| 3.2      | Temel Fikir . . . . .                              | 13        |
| 3.3      | Netlik Kurma . . . . .                             | 13        |
| 3.4      | Stratejik Düşünme . . . . .                        | 14        |
| 3.5      | Risk ve Kısıt Farkındalığı . . . . .               | 14        |
| 3.6      | İşi İlerletme . . . . .                            | 14        |
| 3.7      | Kalite Standardı Belirleme . . . . .               | 15        |
| 3.8      | Entelektüel Dürüstlük . . . . .                    | 15        |
| 3.9      | Bu İfadelerin Profesyonel Etkisi . . . . .         | 15        |
| 3.10     | Teknik ve Mimari Tartışmalarda Kullanımı . . . . . | 16        |
| 3.11     | Sonuç . . . . .                                    | 16        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Yazılım Mühendisliği Yasaları: Basit Açıklama</b>                  | <b>17</b> |
| 4.1      | Giriş . . . . .                                                       | 17        |
| 4.2      | 1. İşler Sandığından Uzun Sürer . . . . .                             | 17        |
| 4.3      | Hofstadter Yasası . . . . .                                           | 17        |
| 4.4      | 2. İş Kendisine Ayrılan Zamanı Doldurur . . . . .                     | 17        |
| 4.5      | Parkinson Yasası . . . . .                                            | 17        |
| 4.6      | 3. Geç Projeye İnsan Eklemek Projeyi Daha da Geciktirebilir . . . . . | 17        |
| 4.7      | Brooks Yasası . . . . .                                               | 17        |
| 4.8      | 4. Ekip Büyüdükçe Kişi Başına Verim Azalabilir . . . . .              | 18        |
| 4.9      | Ringelmann Etkisi . . . . .                                           | 18        |
| 4.10     | 5. Ölçü Hedef Olursa Bozulur . . . . .                                | 18        |
| 4.11     | Goodhart Yasası . . . . .                                             | 18        |
| 4.12     | 6. API'deki Her Görünen Davranışa Birileri Bağımlı Olabilir . . . . . | 18        |
| 4.13     | Hyrum Yasası . . . . .                                                | 18        |
| 4.14     | 7. Organizasyon Yapısı Yazılım Mimarisine Yansır . . . . .            | 18        |
| 4.15     | Conway Yasası . . . . .                                               | 18        |
| 4.16     | 8. Programlar Zamanla Gereksiz Büyüyebilir . . . . .                  | 19        |
| 4.17     | Zawinski Yasası . . . . .                                             | 19        |
| 4.18     | 9. Yanlış Gidebilecek Şeyler Yanlış Gidebilir . . . . .               | 19        |
| 4.19     | Murphy Yasası . . . . .                                               | 19        |
| 4.20     | 10. İnternette Yanlış Cevap Daha Çok Tepki Çeker . . . . .            | 20        |
| 4.21     | Cunningham Yasası . . . . .                                           | 20        |
| 4.22     | 11. Her Şeyin Büyük Kısmı Kalitesiz Olabilir . . . . .                | 20        |
| 4.23     | Sturgeon Yasası . . . . .                                             | 20        |
| 4.24     | 12. Ölçülmesi Gereken Şey Bir Şekilde Ölçülebilir . . . . .           | 20        |
| 4.25     | Gilb Yasası . . . . .                                                 | 20        |
| 4.26     | Teknik Borç Açısından Basit Yorum . . . . .                           | 21        |
| 4.27     | Sonuç . . . . .                                                       | 21        |
| <b>5</b> | <b>LLM Wiki ve AI Destekli Kişisel Bilgi Tabanı</b>                   | <b>22</b> |
| 5.1      | Giriş . . . . .                                                       | 22        |
| 5.2      | Klasik RAG Yaklaşımı . . . . .                                        | 22        |
| 5.3      | LLM Wiki Yaklaşımı . . . . .                                          | 22        |
| 5.4      | Obsidian'ın Rolü . . . . .                                            | 23        |
| 5.5      | Sistem Nasıl Çalışır? . . . . .                                       | 23        |
| 5.6      | Kendi Kendini Geliştiren Bilgi Tabanı Fikri . . . . .                 | 24        |
| 5.7      | Büyük Context Yerine Dosya Tabanlı Hafıza . . . . .                   | 25        |
| 5.8      | Denetlenebilirlik Avantajı . . . . .                                  | 25        |
| 5.9      | Hangi Alanlarda Kullanılabilir? . . . . .                             | 26        |
| 5.10     | Bilgi Geometrisi Açısından Yorum . . . . .                            | 27        |
| 5.11     | Klasik RAG ile Karşılaştırma . . . . .                                | 27        |
| 5.12     | Sınırlamalar . . . . .                                                | 27        |
| 5.13     | Sonuç . . . . .                                                       | 27        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>6</b> | <b>Kültür ve Tolere Edilen Davranışlar</b>                | <b>29</b> |
| 6.1      | Giriş . . . . .                                           | 29        |
| 6.2      | Tolere Etmek Kültür Üretir . . . . .                      | 29        |
| 6.3      | Sözler ve Gerçek Davranış Arasındaki Fark . . . . .       | 29        |
| 6.4      | Kültür Posterle Değişmez . . . . .                        | 30        |
| 6.5      | Kültür Yavaş Değişir . . . . .                            | 30        |
| 6.6      | Kültürü Değiştirmek İçin Sistem Değişmelidir . . . . .    | 31        |
| 6.7      | Yeni Düzene Davet Etmek Yetmez . . . . .                  | 31        |
| 6.8      | Yazılım Ekipleri Açısından Yorum . . . . .                | 32        |
| 6.9      | Sonuç . . . . .                                           | 32        |
| <b>7</b> | <b>Developer Experience Kavramları</b>                    | <b>33</b> |
| 7.1      | 1. Daha İyi Dokümantasyon . . . . .                       | 33        |
| 7.2      | 2. Daha Kolay Local Setup . . . . .                       | 33        |
| 7.3      | 3. Daha Tutarlı Tooling . . . . .                         | 34        |
| 7.4      | 4. Secure by Default . . . . .                            | 35        |
| 7.5      | 5. Shifting Left . . . . .                                | 36        |
| 7.6      | 6. Daha Anlaşılır Modül Sınırları . . . . .               | 37        |
| 7.7      | 7. DDD: Domain-Driven Design . . . . .                    | 37        |
| 7.8      | 8. Daha Hızlı Feedback Loop'ları . . . . .                | 38        |
| 7.9      | Bu Kavramların Ortak Amacı . . . . .                      | 39        |
| 7.10     | Özet . . . . .                                            | 40        |
| 7.11     | Üst Kavram: Engineering Experience Architecture . . . . . | 40        |
| <b>8</b> | <b>Codebase'de Niyet Aktarımı İçin Claude Template'i</b>  | <b>40</b> |
| 8.1      | Amaç . . . . .                                            | 40        |
| 8.2      | Claude'a Verilecek Ana Prompt . . . . .                   | 41        |
| 8.3      | 1. Codebase Amacının Analizi . . . . .                    | 41        |
| 8.4      | 2. Mimari Niyet Analizi . . . . .                         | 41        |
| 8.5      | 3. İsimlendirme Niyeti Denetimi . . . . .                 | 42        |
| 8.6      | 4. Domain Model Denetimi . . . . .                        | 43        |
| 8.7      | 5. Kullanıcı Niyeti Akışlarının Haritalanması . . . . .   | 43        |
| 8.8      | 6. Yan Etki Denetimi . . . . .                            | 45        |
| 8.9      | 7. Davranış Sözleşmelerinin Çıkarılması . . . . .         | 45        |
| 8.10     | 8. Test Niyeti Denetimi . . . . .                         | 46        |
| 8.11     | 9. Davranış Değiştirmeden Refactor Planı . . . . .        | 47        |
| 8.12     | 10. Claude'dan Beklenen Nihai Çıktı Formatı . . . . .     | 47        |
| 8.13     | Kısa Kullanım Template'i . . . . .                        | 48        |
| 8.14     | Refactor Aşaması İçin İkinci Prompt . . . . .             | 49        |
| 8.15     | Önerilen Kullanım Sırası . . . . .                        | 49        |
| 8.16     | Sonuç . . . . .                                           | 50        |

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Codebase’de Niyet Aktarımı ile İlişkili Sektörel Kavramlar</b>             | <b>50</b> |
| 9.1       | Giriş . . . . .                                                               | 50        |
| 9.2       | 1. Intention-Revealing Interface . . . . .                                    | 50        |
| 9.3       | 2. Ubiquitous Language . . . . .                                              | 51        |
| 9.4       | 3. Domain-Driven Design . . . . .                                             | 52        |
| 9.5       | 4. Bounded Context . . . . .                                                  | 52        |
| 9.6       | 5. Self-Documenting Code . . . . .                                            | 53        |
| 9.7       | 6. Architecture Decision Record . . . . .                                     | 53        |
| 9.8       | 7. Behavior Contract . . . . .                                                | 54        |
| 9.9       | 8. Invariant . . . . .                                                        | 55        |
| 9.10      | 9. Business Rule . . . . .                                                    | 55        |
| 9.11      | 10. Acceptance Criteria . . . . .                                             | 56        |
| 9.12      | 11. Executable Specification . . . . .                                        | 56        |
| 9.13      | 12. Regression Test . . . . .                                                 | 56        |
| 9.14      | 13. Design by Contract . . . . .                                              | 57        |
| 9.15      | 14. Semantic Naming . . . . .                                                 | 57        |
| 9.16      | 15. Information Architecture . . . . .                                        | 58        |
| 9.17      | 16. Architectural Boundary . . . . .                                          | 59        |
| 9.18      | 17. Separation of Concerns . . . . .                                          | 59        |
| 9.19      | 18. Codebase Intent Map . . . . .                                             | 60        |
| 9.20      | 19. Intent-Preserving Architecture . . . . .                                  | 60        |
| 9.21      | 20. Intent-Revealing Codebase Design . . . . .                                | 61        |
| 9.22      | Özet . . . . .                                                                | 62        |
| <b>10</b> | <b>Yazılımda Takip Edilebilirlik Geometrisi</b>                               | <b>64</b> |
| 10.1      | Giriş . . . . .                                                               | 64        |
| 10.2      | Anlamak ve Takip Edebilmek Arasındaki Fark . . . . .                          | 64        |
| 10.3      | Agent Açısından Takip Edilebilirlik . . . . .                                 | 65        |
| 10.4      | Kötü Projede Agent’ın Davranışı . . . . .                                     | 65        |
| 10.5      | İyi Projede Agent’ın Davranışı . . . . .                                      | 66        |
| 10.6      | Takip Edilebilirlik Geometrisi . . . . .                                      | 67        |
| 10.7      | Takip Edilebilirlik Borcu . . . . .                                           | 68        |
| 10.8      | Yazılımda Üç Seviye . . . . .                                                 | 68        |
| 10.9      | Sonuç . . . . .                                                               | 69        |
| <b>11</b> | <b>Yazılım Mühendisliğinde Basitlik, Kavramsallaştırma ve Sistem Tasarımı</b> | <b>70</b> |
| 11.1      | Giriş . . . . .                                                               | 70        |
| 11.2      | Yazılım Mühendisliği: Mühendislik ve Zanaatkârlık Arasında . . . . .          | 70        |
| 11.3      | Kavramsallaştırmanın Önemi . . . . .                                          | 70        |
| 11.4      | Kavramsallaştırma Örneği: Online Eğitim Platformu . . . . .                   | 71        |
| 11.5      | Basit Tasarımın Ana Fikri . . . . .                                           | 72        |
| 11.6      | Kent Beck’in Basit Tasarım İçin Dört Kuralı . . . . .                         | 73        |
| 11.7      | Basit Tasarım Kurallarında Öncelik Sırası . . . . .                           | 74        |
| 11.8      | Unix Felsefesi ile Bağlantı . . . . .                                         | 74        |
| 11.9      | Çeviklik ve Basit Tasarım . . . . .                                           | 75        |

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| 11.10     | Sonuç . . . . .                                    | 75        |
| <b>12</b> | <b>Yazılımda Hikâyenin Önemi</b>                   | <b>76</b> |
| 12.1      | Giriş . . . . .                                    | 76        |
| 12.2      | Hikâye Neden Gereklidir? . . . . .                 | 76        |
| 12.3      | Hikâye ile Kavramların Ortaya Çıkması . . . . .    | 76        |
| 12.4      | Hikâye Olmadan Kodun Durumu . . . . .              | 77        |
| 12.5      | Hikâye Varsa Kodun Durumu . . . . .                | 78        |
| 12.6      | Doğru Sıralama . . . . .                           | 78        |
| 12.7      | Hikâyeyi Çıkarmak İçin Sorulacak Sorular . . . . . | 79        |
| 12.8      | Hikâyenin Yazılıma Katkısı . . . . .               | 79        |
| 12.9      | Sonuç . . . . .                                    | 80        |

# 1 Giriş

Modern yazılım geliştirme, yalnızca kod yazma faaliyeti değildir. Bir yazılım sisteminin sürdürülebilirliği; kodun kalitesi kadar, sistemin anlaşılabilirliği, geliştirilebilirliği, test edilebilirliği, dokümanite edilebilirliği ve güvenli biçimde değiştirilebilirliği ile de ilgilidir.

Bu nedenle günümüzde yazılım mühendisliği; teknik borç, Developer Experience, Agent Experience, mimari kararlar, test stratejileri, kültür, iletişim dili ve bilgi yönetimi gibi birçok katmanın birlikte ele alınmasını gerektirir. Özellikle yapay zekâ destekli geliştirme araçlarının yazılım süreçlerine dahil olmasıyla birlikte, bir codebase'in yalnızca insanlar tarafından değil, AI agent'lar tarafından da takip edilebilir hale gelmesi önem kazanmıştır.

Bu çalışmanın temel fikri şudur:

Kötü projede insan ve agent tahmin eder; iyi projede insan ve agent iz sürer.

Bir yazılım projesi, yalnızca çalışan koddan ibaret değildir. Aynı zamanda kararların, niyetlerin, sınırların, davranış beklentilerinin, testlerin, dokümantasyonun ve operasyonel pratiklerin oluşturduğu daha geniş bir sistemdir. Bu sistem sağlıklı kurulmadığında teknik borç yalnızca kodda değil; mimaride, testlerde, dokümantasyonda, kültürde, feedback loop'larında ve geliştirici deneyiminde de birikir.

Bu doküman, yazılım geliştirme süreçlerinde ortaya çıkan bu borç türlerini daha görünür hale getirmeyi amaçlar. Teknik borç takip listesi, profesyonel iletişim dili, yazılım mühendisliği yasaları, LLM destekli bilgi tabanları, organizasyon kültürü, Claude Code komutları ve Developer Experience kavramları bu çerçevede ele alınmaktadır.

Amaç, yazılım mühendisliğini yalnızca implementasyon düzeyinde değil; anlam, takip edilebilirlik, karar kalitesi, sistem sağlığı ve insan-agent iş birliği düzeyinde değerlendirebilecek bir bakış açısı geliştirmektir.

## 2 Teknik Borç Takip Listesi

### 2.1 1. Kod Geometrisi Borcu

- Modül sınırları belirsiz mi?
- UI, service, store, API ve helper katmanları birbirine karışmış mı?
- Veri akışı tek yönlü mü, yoksa her yer her yere bağlı mı?
- Döngüsel bağımlılıklar var mı?
- God component veya God service oluşmuş mu?
- Bir dosya veya class çok fazla sorumluluk taşıyor mu?
- Helper veya util dosyaları domain logic saklıyor mu?
- Kod tree, flow, layered yapı yerine spaghetti/network karmaşasına mı dönüşmüş?

### 2.2 2. Davranış Geometrisi Borcu

- Sistemin hangi durumda nasıl davranacağı açık mı?
- Loading, empty, error ve success state'leri tanımlı mı?
- Edge case'ler düşünülmüş mü?
- Kullanıcı yanlış veri girerse sistem ne yapıyor?
- API başarısız olursa sistem nasıl toparlanıyor?
- Retry, timeout ve reconnect davranışları belli mi?
- Permission ve authorization davranışları net mi?
- Session expiry durumunda kullanıcı akışı doğru mu?

### 2.3 3. Spec Borcu

- Özelliğin ne yapacağı açıkça yazılmış mı?
- Acceptance criteria var mı?
- API contract belli mi?
- UI state'leri önceden tarif edilmiş mi?
- Test edilebilir davranış maddeleri var mı?
- “Bu feature tamamlandı” demek için ölçülebilir kriterler var mı?
- AI agent'a verildiğinde yanlış anlaşılmayacak kadar net mi?
- Spec kodla birlikte güncel tutuluyor mu?

## 2.4 4. Test Borcu

- Kritik davranışlar unit test ile korunuyor mu?
- Component testleri var mı?
- Integration testleri gerçek akışları kapsıyor mu?
- E2E testleri temel kullanıcı senaryolarını doğruluyor mu?
- Error-state testleri var mı?
- Empty-state testleri var mı?
- Loading-state testleri var mı?
- Authorization ve permission testleri var mı?
- Retry, timeout ve offline/reconnect testleri var mı?
- Regression testleri eski bug'ların tekrar dönmesini engelliyor mu?
- Mutation test veya benzeri yöntemlerle testlerin gerçekten bug yakaladığı kontrol ediliyor mu?
- Testler hızlı çalışıyor mu?
- Testler flaky mi?

## 2.5 5. Developer Experience Borcu

- Proje local ortamda kolay ayağa kalkıyor mu?
- README güncel mi?
- Kurulum adımları açık mı?
- Environment variable'lar belgelenmiş mi?
- Mock data veya seed data mekanizması var mı?
- Testleri çalıştırmak kolay mı?
- Build ve lint komutları net mi?
- Yeni bir geliştirici projeye hızlı adapte olabilir mi?
- Tooling tutarlı mı?
- Hata mesajları geliştiriciye gerçekten yardımcı oluyor mu?



## 2.6 6. Agent Experience Borcu

- AI agent codebase'i okuyunca mimariyi anlayabilir mi?
- Dosya ve klasör isimleri niyeti açıkça gösteriyor mu?
- Domain kavramları tutarlı mı?
- Spec, test ve kod arasında bağlantı var mı?
- Agent hangi dosyada değişiklik yapacağını anlayabilir mi?
- Çok fazla gizli yan etki var mı?
- Kodda magic string, magic number veya belirsiz convention var mı?
- Agent'ın güvenle değişiklik yapabileceği küçük modüller var mı?
- Testler agent'ın yaptığı değişikliği hızlı doğrulayabiliyor mu?

## 2.7 7. Dokümantasyon Borcu

- Mimari kararlar belgelenmiş mi?
- Neden böyle yapıldığı yazıyor mu?
- API dokümantasyonu güncel mi?
- Domain kavramları açıklanmış mı?
- Kritik akışların diagramları var mı?
- Setup, deploy, test ve debug süreçleri yazılı mı?
- Dokümantasyon koddan kopmuş mu?
- Eski ve yanlış dokümanlar temizlenmiş mi?

## 2.8 8. Domain ve Anlam Borcu

- Aynı kavram farklı yerlerde farklı isimlerle mi geçiyor?
- Business dili ile kod dili uyumlu mu?
- Entity, model, DTO, state ve API isimleri tutarlı mı?
- Kavramların sınırları net mi?
- Domain logic doğru yerde mi duruyor?
- Kod, ürünün gerçek anlamını yansıtıyor mu?
- Yeni biri domain'i koddan okuyarak anlayabilir mi?

## 2.9 9. Mimari Borç

- Katmanlar doğru ayrılmış mı?
- Bağımlılık yönleri kontrol altında mı?
- Infrastructure kodu domain logic'e sızmış mı?
- UI business logic taşıyor mu?
- Service'ler çok fazla orchestration yapıyor mu?
- State yönetimi merkezi mi, dağınık mı?
- Feature bazlı sınırlar net mi?
- Refactor yapmadan yeni feature eklemek zorlaşıyor mu?

## 2.10 10. Güvenlik ve Privacy Borcu

- Authentication akışı güvenli mi?
- Authorization kuralları açık mı?
- Sensitive data gereksiz loglanıyor mu?
- Token yönetimi doğru mu?
- Secure by default yaklaşımı var mı?
- Input validation yeterli mi?
- API endpoint'leri gereksiz veri döndürüyor mu?
- KVKK/GDPR gibi privacy gereklilikleri düşünülmüş mü?

## 2.11 11. Operasyonel Borç

- Deploy süreci güvenilir mi?
- Rollback mekanizması var mı?
- Monitoring ve logging yeterli mi?
- Hata olduğunda sebebi bulunabiliyor mu?
- Environment farkları sorun çıkarıyor mu?
- Config değerleri belgelenmiş mi?
- Servisler arası bağlantılar gözlemlenebilir mi?
- Disaster recovery planı var mı?

## 2.12 12. Feedback Loop Borcu

- Kod değişikliğinin doğru olup olmadığı hızlı anlaşılabilir mi?
- Testler çok uzun sürdüğü için çalıştırılmıyor mu?
- Build yavaş mı?
- Lint ve type check erken hata yakalıyor mu?
- CI pipeline güvenilir mi?
- Developer değişiklik sonrası sonucu hemen görebiliyor mu?
- AI agent yaptığı değişikliği hızlı doğrulayabiliyor mu?

## 2.13 Teknik Borç Kaydı İçin Önerilen Format

Her teknik borç maddesini şu formatta takip ederdim:

- **Başlık:** Borcun kısa adı.
- **Kategori:** Kod geometrisi, test, spec, DevEx, güvenlik vb.
- **Belirti:** Bu borç kendini nasıl gösteriyor?
- **Kök neden:** Bu borç neden oluştu?
- **Etkisi:** Hangi geliştirme, test veya kullanıcı akışını zorlaştırıyor?
- **Risk seviyesi:** Düşük, orta, yüksek, kritik.
- **Görünür maliyet:** Bug, yavaş geliştirme, onboarding zorluğu, güvenlik riski vb.
- **AI/Agent etkisi:** Agent bu borç yüzünden nerede hata yapabilir?
- **Çözüm önerisi:** Nasıl düzeltilmeli?
- **Öncelik:** Şimdi, yakında, sonra.
- **DoD:** Bu borcun kapandığını nasıl anlayacağız?

## 2.14 Örnek Teknik Borç Kaydı

**Başlık:**

CommercialPage component çok fazla sorumluluk taşıyor.

**Kategori:**

Kod geometrisi / mimari borç / test borcu

**Belirti:**

Component hem UI rendering, hem API çağrısı, hem state yönetimi, hem error handling, hem de payload hazırlama yapıyor.

Kök neden:

İlk geliştirme hızlı yapılmış, sonra feature'lar aynı dosyaya eklenmiş.

Etkisi:

Yeni feature eklemek zorlaşıyor.

Test yazmak zorlaşıyor.

AI agent değişiklik yaparken yanlış alanları bozabilir.

Risk seviyesi:

Yüksek

AI/Agent etkisi:

Agent küçük bir UI değişikliği yaparken business logic'i bozabilir.

Çözüm:

UI, container, service, state ve helper sorumlulukları ayrılmalı.

Magic string'ler constants dosyasına taşınmalı.

Payload hazırlama ayrı helper/service olmalı.

DoD:

Component yalnızca rendering ve event forwarding yapmalı.

Business logic service/facade katmanına taşınmalı.

Critical flow component/integration test ile korunmalı.

## 3 Dil Kullanımı ve Profesyonel Güvenilirlik

### 3.1 Giriş

Profesyonel ortamlarda güvenilirlik yalnızca kişinin sahip olduğu bilgiyle belirlenmez. Aynı zamanda bu bilginin nasıl ifade edildiği, hangi cümlelerle çerçevelendiği ve konuşmanın hangi yöne taşındığı da kişinin algılanan etkisini doğrudan belirler.

Bir kişi doğru fikirlere sahip olabilir; ancak bu fikirleri dağınık, savunmacı veya belirsiz bir dille ifade ederse beklenen etkiyi oluşturamayabilir. Buna karşılık, daha stratejik ve net bir dil kullanımı, kişinin toplantı içindeki ağırlığını, güvenilirliğini ve liderlik etkisini artırabilir.

Bu bağlamda belirli ifadeler, yalnızca iletişim aracı değil; aynı zamanda profesyonel duruş göstergesidir.

### 3.2 Temel Fikir

Bu yaklaşımın ana fikri şudur:

Kullanılan dil, kişinin profesyonel güvenilirliğini ve odadaki etkisini şekillendirir.

Başka bir ifadeyle, insanlar yalnızca ne söylendiğine değil, nasıl söylendiğine de tepki verir. Bazı cümleler konuşmacının daha net, daha stratejik, daha dikkatli ve daha güvenilir algılanmasını sağlar.

### 3.3 Netlik Kurma

Bazı ifadeler karmaşık bir tartışmayı sadeleştirmek ve asıl meseleyi görünür hale getirmek için kullanılır:

- “Let’s simplify this.”
- “Let’s get clear on the real issue.”

Bu tür ifadeler, konuşmacının karmaşa içinde kaybolmadığını ve asıl problemi ayıklayabildiğini gösterir. Özellikle teknik, stratejik veya organizasyonel tartışmalarda bu beceri son derece değerlidir.

Netlik kurabilen kişi, yalnızca fikir üreten değil; düşüncüyü düzenleyen kişidir.

### 3.4 Stratejik Düşünme

Bazı ifadeler tartışmayı detay seviyesinden çıkarıp hedef, sonuç ve büyük resim düzeyine taşır:

- “What outcome are we driving?”
- “What outcome are we optimizing for?”
- “Let’s zoom out for a second.”

Bu ifadeler, konuşmacının yalnızca mevcut problemi değil, problemin hangi sonuç için çözüldüğünü de düşündüğünü gösterir. Böylece tartışma yalnızca görev odaklı olmaktan çıkar, amaç odaklı hale gelir.

Stratejik dil, yapılan iş ile ulaşılacak istenen sonuç arasındaki bağı görünür kılar.

### 3.5 Risk ve Kısıt Farkındalığı

Güvenilir bir profesyonel yalnızca fırsatları değil, riskleri ve sınırlamaları da fark eder. Bu nedenle aşağıdaki ifadeler güçlü bir sahiplenme ve öngörü sinyali verir:

- “Here’s the risk we’re missing.”
- “Here’s the constraint we can’t ignore.”

Bu tür cümleler, konuşmacının yalnızca olumlu senaryoya odaklanmadığını; aynı zamanda sistemin kırılabileceği, yavaşlayabileceği veya başarısız olabileceği noktaları da düşündüğünü gösterir.

Riskleri ifade etmek, karamsarlık değil; sorumluluk bilincidir.

### 3.6 İş İlerletme

Bazı tartışmalar uzun süre soyut düzeyde kalabilir. Bu durumda konuşmayı uygulanabilir bir adıma indirmek önemlidir:

- “What’s the next smallest step?”

Bu ifade, büyük ve karmaşık bir hedefi daha küçük, uygulanabilir ve ölçülebilir bir adıma dönüştürür. Böylece tartışma yalnızca fikir alışverişi olarak kalmaz; eyleme yönelir.

İyi iletişim yalnızca düşünce üretmez; hareket de üretir.

### 3.7 Kalite Standardı Belirleme

Bazı ifadeler ekibin yalnızca işi bitirmesine değil, işi hangi kalite seviyesinde bitirmesi gerektiğine odaklanır:

- “What does great look like here?”
- “What would great look like here?”

Bu cümleler, sıradan bir tamamlanma anlayışının ötesine geçer. Yapılan işin gerçekten başarılı, etkili ve kaliteli olup olmadığını sorgular.

Kalite standardı sorulmadığında, ekip çoğu zaman yalnızca tamamlamaya odaklanır.

### 3.8 Entelektüel Dürüstlük

Profesyonel güvenilirliği artıran en önemli unsurlardan biri de kişinin kendi düşüncesini sorgulayabilmesidir. Bu amaçla kullanılan bazı ifadeler şunlardır:

- “Here’s where I might be wrong.”
- “Let’s challenge that assumption.”

Bu ifadeler, konuşmacının dogmatik bir tavır almadığını gösterir. Kendi varsayımlarını ve düşüncelerini test etmeye açık olmak, güvenilirliği azaltmaz; aksine artırır.

Entelektüel dürüstlük, kişinin haklı çıkmaya değil, doğruyu bulmaya çalıştığını gösterir.

### 3.9 Bu İfadelerin Profesyonel Etkisi

Bu tür cümleler kişiye şu profesyonel nitelikleri kazandırır:

- netlik,
- odak,
- stratejik düşünme,
- risk farkındalığı,
- önceliklendirme becerisi,
- kalite bilinci,
- eleştirel düşünme,
- entelektüel dürüstlük,
- sakın ve güvenilir liderlik etkisi.

Dolayısıyla mesele yalnızca daha etkileyici konuşmak değildir. Asıl mesele, dili kullanarak düşünceyi daha açık, daha güvenilir ve daha yönlendirici hale getirmektir.

### 3.10 Teknik ve Mimari Tartışmalarda Kullanımı

Bu ifadeler özellikle teknik borç, yazılım mimarisi, test stratejisi, ürün geliştirme ve yapay zekâ destekli geliştirme süreçlerinde de kullanılabilir.

Örneğin:

- “Let’s get clear on the real issue” ifadesi, problemi doğru tanımlamak için kullanılabilir.
- “Here’s the risk we’re missing” ifadesi, mimari veya operasyonel riskleri görünür hale getirmek için kullanılabilir.
- “What’s the next smallest step” ifadesi, büyük bir refactor veya teknik borç azaltma sürecini uygulanabilir hale getirmek için kullanılabilir.
- “Let’s challenge that assumption” ifadesi, yanlış kabul edilmiş teknik kararları yeniden değerlendirmek için kullanılabilir.

Bu bakımdan söz konusu ifadeler yalnızca toplantı dili değildir. Aynı zamanda teknik karar alma süreçlerinde de düşünceyi disipline eden araçlardır.

### 3.11 Sonuç

Profesyonel güvenilirlik, yalnızca bilgi sahibi olmakla oluşmaz. Bilginin nasıl ifade edildiği, tartışmanın nasıl çerçevelendiği ve konuşmanın hangi yöne taşındığı da belirleyicidir.

Doğru ifadeler, kişinin yalnızca konuşan biri değil; netleştiren, önceliklendiren, riskleri gören, kalite standardı koyan ve süreci ilerleten biri olarak algılanmasını sağlar.

Dil yalnızca iletişim aracı değildir; profesyonel düşüncenin ve liderlik etkisinin görünür biçimidir.



## 4 Yazılım Mühendisliği Yasaları: Basit Açıklama

### 4.1 Giriş

Yazılım projeleri yalnızca kod yüzünden zorlaşmaz. Çoğu zaman asıl sorun; zaman tahmini, ekip yapısı, iletişim, ölçüm, ürün kapsamı ve risk yönetimi gibi konulardan çıkar.

Bu nedenle yazılım mühendisliğinde bazı bilinen yasalar vardır. Bu yasalar, projelerde tekrar tekrar görülen problemleri basit şekilde anlatır.

### 4.2 1. İşler Sandığından Uzun Sürer

#### 4.3 Hofstadter Yasası

Bu yasa şunu söyler:

Bir iş, beklediğinden daha uzun sürer; bunu hesaba katsan bile.

Yazılımda bir işi başta kolay sanabiliriz. Fakat sonra test, hata, edge case, review, deploy veya entegrasyon problemleri çıkar.

**Ders:** Tahmin yaparken her zaman belirsizlik payı bırakmak gerekir.

### 4.4 2. İş Kendisine Ayrılan Zamanı Doldurur

#### 4.5 Parkinson Yasası

Bu yasa şunu söyler:

İş, kendisine verilen zamanı dolduracak şekilde genişler.

Bir işe iki hafta verilirse, iş çoğu zaman iki haftaya yayılır. Aynı iş daha kısa sürede de yapılabilecek olsa bile, ayrılan zaman işi büyütebilir.

**Ders:** İşleri küçük parçalara bölmek ve net zaman sınırları koymak gerekir.

### 4.6 3. Geç Projeye İnsan Eklemek Projeyi Daha da Geciktirebilir

#### 4.7 Brooks Yasası

Bu yasa şunu söyler:

Geç kalan bir yazılım projesine insan eklemek, projeyi daha da geciktirebilir.

Çünkü yeni gelen kişilerin projeyi öğrenmesi gerekir. Mevcut ekip onları eğitir. Toplantılar ve iletişim maliyeti artar.

**Ders:** Geciken projeye hemen yeni insan eklemek yerine, önce gecikmenin gerçek nedeni bulunmalıdır.

## 4.8 4. Ekip Büyüdükçe Kişi Başına Verim Azalabilir

### 4.9 Ringelmann Etkisi

Bu etki şunu anlatır:

Grup büyüdükçe, bireysel katkı ve verim düşebilir.

Küçük ekiplerde herkesin katkısı daha görünürdür. Büyük ekiplerde ise koordinasyon zorlaşır, sorumluluk dağılır ve verim azalabilir.

**Ders:** Ekip büyüklüğü dikkatli yönetilmelidir.

## 4.10 5. Ölçü Hedef Olursa Bozulur

### 4.11 Goodhart Yasası

Bu yasa şunu söyler:

Bir ölçü hedef haline geldiğinde, iyi bir ölçü olmaktan çıkar.

Örneğin sadece test coverage hedeflenirse, ekip kaliteli test yerine sadece coverage artıran zayıf testler yazabilir.

Benzer şekilde sadece commit sayısı ölçülürse, insanlar gereksiz commit atmaya başlayabilir.

**Ders:** Metric tek başına kalite göstergesi değildir. Ölçümler bağlamla birlikte değerlendirilmelidir.

## 4.12 6. API'deki Her Görünen Davranışa Birileri Bağımlı Olabilir

### 4.13 Hyrum Yasası

Bu yasa şunu söyler:

Yeterince kullanıcısı olan bir API'nin gözlemlenebilir her davranışına birileri bağımlı olur.

Sen bir davranış resmi contract içinde tanımlamamış olabilirsin. Fakat başka bir ekip veya kullanıcı o davranış fark edip kullanmış olabilir. Sen bunu değiştirdiğinde onların sistemi bozulabilir.

**Ders:** API değişiklikleri dikkatli yapılmalıdır. Geriye uyumluluk önemlidir.

## 4.14 7. Organizasyon Yapısı Yazılım Mimarisine Yansır

### 4.15 Conway Yasası

Bu yasa şunu söyler:

Organizasyonlar, kendi iletişim yapılarına benzeyen sistemler üretir.

Eğer ekipler birbirinden kopuk çalışıyorsa, yazılım mimarisi de kopuk hale gelir. Frontend, backend, infra ve ürün ekipleri arasında iletişim zayıfsa, sistemde de uyumsuzluklar oluşur.

**Ders:** İyi mimari için iyi ekip iletişimi gerekir.

## 4.16 8. Programlar Zamanla Gereksiz Büyüyebilir

### 4.17 Zawinski Yasası

Bu yasa biraz esprili biçimde şunu söyler:

Her program, zamanla genişleyerek daha fazla özellik eklemeye çalışır.

Basit başlayan bir uygulama zamanla çok fazla özellik alabilir. Bu da sistemi karmaşıktırır.

**Ders:** Her özelliği eklemek doğru değildir. Ürünün sınırları korunmalıdır.

## 4.18 9. Yanlış Gidebilecek Şeyler Yanlış Gidebilir

### 4.19 Murphy Yasası

Bu yasa şunu söyler:

Yanlış gidebilecek bir şey varsa, bir noktada yanlış gidecektir.

Yazılımda API çökebilir, network kopabilir, kullanıcı yanlış veri girebilir, token expire olabilir veya sistem beklenmedik hata verebilir.

**Ders:** Sadece başarılı senaryo düşünülmemelidir. Hata durumları da tasarlanmalıdır.

## 4.20 10. İnternette Yanlış Cevap Daha Çok Tepki Çeker

### 4.21 Cunningham Yasası

Bu yasa şunu söyler:

İnternette doğru cevabı almanın en hızlı yolu soru sormak değil, yanlış cevap yazmaktır.

İnsanlar yanlış bilgiyi düzeltmeye daha hızlı tepki verir. Bu yasa daha çok insan davranışıyla ilgilidir.

**Ders:** Feedback mekanizmaları yazılım geliştirmede çok değerlidir.

## 4.22 11. Her Şeyin Büyük Kısmı Kalitesiz Olabilir

### 4.23 Sturgeon Yasası

Bu yasa şunu söyler:

Her şeyin büyük bir kısmı düşük kalitededir.

Bu ifade biraz abartılıdır; fakat yazılım için önemli bir uyarı taşır. Her kod, her doküman, her araç veya her AI çıktısı otomatik olarak kaliteli değildir.

**Ders:** Kalite kontrol, review ve eleme süreci gereklidir.

## 4.24 12. Ölçülmesi Gereken Şey Bir Şekilde Ölçülebilir

### 4.25 Gilb Yasası

Bu yasa şunu söyler:

Ölçülmesi gereken bir şey, hiç ölçmemekten daha iyi bir yöntemle ölçülebilir.

Bazı şeyleri tam olarak ölçmek zor olabilir. Fakat yine de yaklaşık, pratik veya dolaylı bir ölçüm yapmak mümkündür.

**Ders:** Kalite, hız, güvenilirlik ve teknik borç gibi konular tamamen ölçülemez diye bırakılmamalıdır.

## 4.26 Teknik Borç Açısından Basit Yorum

Bu yasalar teknik borç takibinde şu şekilde kullanılabilir:

- İşler sürekli uzuyorsa, tahmin ve planlama borcu olabilir.
- Yeni kişi gelince proje yavaşlıyorsa, onboarding veya dokümantasyon borcu olabilir.
- Test coverage yüksek ama bug çoksa, yanlış metric borcu olabilir.
- API değişince sistem bozuluyorsa, contract ve compatibility borcu olabilir.
- Ekipler kopuk çalışıyorsa, organizasyon ve mimari borç oluşabilir.
- Ürün sürekli şişiyorsa, scope creep borcu olabilir.
- Hata durumları düşünülmemişse, resilience ve error handling borcu olabilir.

## 4.27 Sonuç

Bu yasalar, yazılım projelerinde sık görülen problemleri basit şekilde anlatır. Ana fikir şudur:

Yazılım geliştirme yalnızca kod yazma işi değildir; zaman, ekip, iletişim, ölçüm, mimari ve risk yönetimi işidir.

Bu yüzden iyi bir engineering leader yalnızca kod kalitesine değil; ekip yapısına, ölçümlere, geri bildirim döngülerine, API sözleşmelerine ve proje kapsamına da dikkat etmelidir.

## 5 LLM Wiki ve AI Destekli Kişisel Bilgi Tabanı

### 5.1 Giriş

LLM Wiki yaklaşımı, yapay zekâyı yalnızca soru cevaplayan bir araç olarak değil, bilgiyi düzenleyen, sınıflandıran, özetleyen ve ilişkilendiren bir bilgi asistanı olarak kullanma fikrine dayanır.

Bu yaklaşımda temel amaç, tüm bilgiyi her seferinde büyük bir prompt içine doldurmak değildir. Bunun yerine, ham kaynaklar okunabilir, denetlenebilir ve güncellenebilir bir Markdown bilgi tabanına dönüştürülür.

Bu bilgi tabanı daha sonra insan veya AI agent tarafından tekrar tekrar kullanılabilir.

LLM Wiki yaklaşımı, yapay zekânın ham bilgiyi düzenli bir bilgi mimarisine dönüştürmesi fikridir.

### 5.2 Klasik RAG Yaklaşımı

Klasik RAG sistemlerinde belgeler genellikle küçük parçalara ayrılır, embedding'leri çıkarılır ve bir vector database içine yerleştirilir. Kullanıcı soru sorduğunda sistem, ilgili parçaları bulur ve LLM'e gönderir.

Bu süreç basitçe şöyle gösterilebilir:

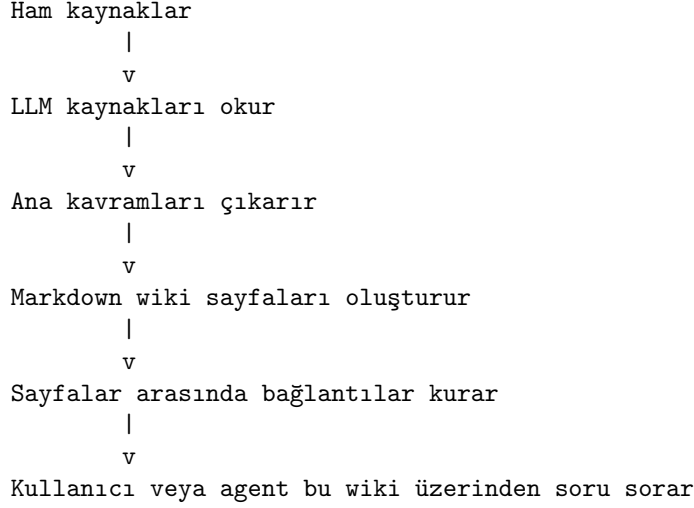
```
PDF / Doküman / Notlar
    |
    v
Chunk'lara bölme
    |
    v
Embedding çıkarma
    |
    v
Vector database'e kaydetme
    |
    v
Soru gelince ilgili parçaları bulma
    |
    v
LLM ile cevap üretme
```

Bu yöntem özellikle büyük veri koleksiyonları için güçlüdür. Ancak kişisel bilgi tabanları, araştırma notları veya proje hafızası gibi alanlarda bazen gereğinden fazla karmaşık olabilir.

### 5.3 LLM Wiki Yaklaşımı

LLM Wiki yaklaşımında ise bilgi önce yapılandırılmış Markdown sayfalarına dönüştürülür. Yani yapay zekâ yalnızca ham veriden cevap üretmez; önce bilgiyi

düzenler, başlıklandırır, özetler ve bağlantılı bir wiki yapısı oluşturur.  
Bu süreç şöyle düşünülebilir:



Bu yaklaşımda bilgi, yalnızca modelin context penceresinde geçici olarak duran bir şey değildir. Bilgi, dosya sistemi içinde yaşayan, okunabilen ve güncellenebilen bir yapıya dönüşür.

## 5.4 Obsidian’ın Rolü

Obsidian, Markdown dosyaları üzerine kurulu bir not alma ve bilgi yönetimi aracıdır. LLM Wiki yaklaşımında Obsidian, bilgi tabanının görsel ve düzenlenebilir arayüzü gibi çalışır.

Bu yapı şu şekilde düşünülebilir:

|          |                                |
|----------|--------------------------------|
| Obsidian | = Bilgi tabanının IDE’si       |
| Markdown | = Bilginin dosya formatı       |
| LLM      | = Bilgiyi düzenleyen asistan   |
| Wiki     | = Yapılandırılmış bilgi tabanı |

Bu benzetme özellikle yazılım geliştiriciler için güçlüdür. Çünkü burada bilgi, kod gibi dosyalara ayrılır; dosyalar arasında bağlantılar kurulur; içerik zamanla düzenlenir ve geliştirilebilir hale gelir.

Obsidian bilgi tabanının çalışma alanı, Markdown dosyaları bilgi yapısı, LLM ise bu yapıyı düzenleyen asistan olarak düşünülebilir.

## 5.5 Sistem Nasıl Çalışır?

Bu yaklaşımda temel akış oldukça sadedir:

1. Ham kaynaklar bir klasöre eklenir.

2. LLM bu kaynakları okur.
3. Ana kavramları ve önemli bilgileri çıkarır.
4. Her kavram için Markdown sayfaları oluşturur.
5. İlgili sayfalar arasında bağlantılar kurar.
6. Eksik, tekrar eden veya dağınık bilgileri düzenler.
7. Kullanıcının sorularını bu wiki üzerinden cevaplar.
8. Gerekirse grafik, slayt, özet veya yeni doküman üretip tekrar sisteme yazar.

Bu yapı sayesinde bilgi tabanı statik bir arşiv olmaktan çıkar. Yeni kaynaklar geldikçe büyüyen, düzenlenen ve gelişen canlı bir sistem haline gelir.

## 5.6 Kendi Kendini Geliştiren Bilgi Tabanı Fikri

Bu yaklaşım bazen “kendi kendini geliştiren ikinci beyin” gibi anlatılır. Bu ifade biraz iddialı olsa da temel fikir anlaşılırdır.

Normalde bir kişi not tutarken şu işleri manuel olarak yapar:

- kaynakları okuma,
- önemli noktaları seçme,
- başlıklandırma,
- özetleme,
- etiketleme,
- kavramları ayırma,
- ilgili notları birbirine bağlama,
- tekrarları temizleme,
- eski bilgi ile yeni bilgiyi ilişkilendirme.

LLM Wiki yaklaşımında bu işlerin önemli bir kısmı yapay zekâ tarafından yapılabilir. Böylece kullanıcı yalnızca kaynakları sisteme ekler; LLM ise bu kaynakları daha düzenli ve kullanılabilir bir bilgi mimarisine dönüştürür.



## 5.7 Büyük Context Yerine Dosya Tabanlı Hafıza

Bu yaklaşımın en önemli fikri şudur:

Her şeyi tek bir büyük prompt içine doldurmak yerine, yapay zekânın okuyup yazabildiği düzenli bir dosya sistemi oluşturmak gerekir.

Bu bakış açısında context penceresi, yapay zekânın tek hafızası olmak zorunda değildir. Bunun yerine, bilgi dosyalarda yaşar. LLM gerektiğinde bu dosyaları okur, düzenler, yeni dosyalar üretir ve bilgi tabanını günceller.

Bu yapı şöyle gösterilebilir:

```
Kaynaklar
|
v
Markdown Wiki
|
v
Agent Sorgusu
|
v
Yeni Çıktılar
|
v
Wiki'ye Geri Yazma
```

Bu yaklaşım daha ucuz, daha denetlenebilir ve daha sürdürülebilir olabilir. Çünkü Markdown dosyaları insan tarafından da doğrudan okunabilir.

## 5.8 Denetlenebilirlik Avantajı

Vector database içinde saklanan embedding'ler insan tarafından doğrudan okunamaz. Buna karşılık Markdown dosyaları açıktır, basittir ve kontrol edilebilir.

Bu nedenle LLM Wiki yaklaşımı şu avantajları sağlar:

- bilgi insan tarafından okunabilir,
- yanlış bilgi daha kolay fark edilir,
- sayfalar manuel olarak düzenlenebilir,
- kaynaklar ve özetler karşılaştırılabilir,
- bilgi yapısı zamanla iyileştirilebilir,
- karmaşık altyapı ihtiyacı azalabilir.

Bu özellikler, özellikle kişisel bilgi yönetimi ve küçük-orta ölçekli araştırma sistemleri için önemlidir.

## 5.9 Hangi Alanlarda Kullanılabilir?

LLM Wiki yaklaşımı birçok alanda kullanılabilir:

- araştırma notları,
- akademik makale okuma,
- teknik dokümantasyon,
- proje hafızası,
- kod tabanı açıklamaları,
- toplantı notları,
- ürün ve strateji bilgisi,
- kişisel öğrenme sistemi,
- teknik borç kayıtları,
- mimari karar kayıtları.

Örneğin bir yazılım projesinde konuşmalar, kararlar, teknik notlar ve dökümanlar sisteme verilebilir. LLM bunları şu tür sayfalara dönüştürebilir:

- Teknik Borç,
- Spec-Driven Development,
- Agent Experience,
- Developer Experience,
- Kod Geometrisi,
- Mimari Kararlar,
- Test Stratejisi,
- Ürün Davranışı.

Böylece dağınık bilgi, daha sonra sorgulanabilecek düzenli bir bilgi tabanına dönüşür.

## 5.10 Bilgi Geometrisi Açısından Yorum

Bu yaklaşım, bilgi geometrisi açısından da anlamlıdır. Çünkü burada yapılan şey yalnızca bilgiyi depolamak değildir. Asıl yapılan şey, bilginin ilişkisel yapısını kurmaktır.

Yani LLM şu işi yapar:

Ham bilgiyi alır, kavramlara ayırır, kavramlar arasında bağlantılar kurar ve ortaya okunabilir bir bilgi geometrisi çıkarır.

Bu açıdan LLM Wiki yaklaşımı, bilginin mimarisini kurma yöntemidir. Bilgi yalnızca yığılmış dokümanlardan oluşmaz; bağlantılı, sorgulanabilir ve geliştirilebilir bir yapıya dönüşür.

## 5.11 Klasik RAG ile Karşılaştırma

| Klasik RAG                                   | LLM Wiki Yaklaşımı                                      |
|----------------------------------------------|---------------------------------------------------------|
| Belgeler chunk'lara bölünür.                 | Belgeler Markdown wiki sayfalarına dönüştürülür.        |
| Bilgi vector database içinde saklanır.       | Bilgi okunabilir dosyalar içinde saklanır.              |
| Embedding ve retrieval ön plandadır.         | Yapılandırma, özetleme ve bağlantı kurma ön plandadır.  |
| İnsan için denetim daha zordur.              | İnsan için denetim daha kolaydır.                       |
| Büyük veri koleksiyonları için güçlüdür.     | Kişisel ve orta ölçekli bilgi tabanları için pratiktir. |
| Cevap üretimi retrieval kalitesine bağlıdır. | Cevap üretimi düzenlenmiş bilgi mimarisine dayanır.     |

## 5.12 Sınırlamalar

Bu yaklaşım güçlü olsa da her durumda klasik RAG'in yerine geçmez. Özellikle çok büyük kurumsal arşivlerde, milyonlarca belge içeren sistemlerde, karmaşık erişim izinlerinde ve yüksek ölçekli arama gereksinimlerinde klasik RAG, search index ve veritabanı tabanlı çözümler hâlâ gerekli olabilir.

Bu nedenle daha doğru değerlendirme şudur:

LLM Wiki yaklaşımı, RAG'in tamamen yerine geçen bir sistem değil; özellikle kişisel ve denetlenebilir bilgi tabanları için daha sade bir alternatiftir.

## 5.13 Sonuç

LLM Wiki yaklaşımı, yapay zekâyı yalnızca cevap üreten bir chatbot olarak değil, bilgiyi düzenleyen, geliştiren ve bağlam oluşturan bir bilgi asistanı olarak konumlandırır.

Bu yaklaşımda amaç, her şeyi büyük bir context penceresine doldurmak değildir. Amaç, yapay zekânın okuyup yazabildiği temiz, düzenli ve denetlenebilir bir bilgi tabanı oluşturmaktır.

AI'ya her seferinde dev bir prompt vermek yerine, AI'nın düzenleyip güncel tuttuğu bir Markdown bilgi tabanı kurmak daha sürdürülebilir olabilir.

Bu nedenle LLM Wiki yaklaşımı, özellikle kişisel araştırma, yazılım projesi hafızası, teknik dokümantasyon ve öğrenme sistemleri için güçlü bir model sunar.

En kısa ifadeyle:

LLM Wiki, bilginin geometrisini yapay zekâya çizdirip, sonra bu geometri üzerinden tekrar düşünme yöntemidir.

## 6 Kültür ve Tolere Edilen Davranışlar

### 6.1 Giriş

Bir organizasyonun kültürü yalnızca yazılı değerlerden, vizyon cümlelerinden veya eğitimlerden oluşmaz. Kültür, günlük hayatta hangi davranışların kabul edildiği, hangi davranışların görmezden gelindiği ve hangi davranışların sonuç doğurduğu ile şekillenir.

Bu nedenle şu ifade kültürün doğasını güçlü biçimde özetler:

Neyi tolere ediyorsan, o artık kültürünün bir parçasıdır.

Bir davranış sürekli tekrar ediyor ve buna rağmen müdahale edilmiyorsa, o davranış artık istisna değil, organizasyonun fiilî normu haline gelir.

### 6.2 Tolere Etmek Kültür Üretir

Kültür yalnızca aktif olarak yapılan şeylerle değil, görmezden gelinen şeylerle de oluşur. Bir organizasyon belirli davranışları açıkça onaylamasa bile, bu davranışlara sürekli izin veriyorsa, çalışanlar gerçek standardın ne olduğunu bu sessizlikten öğrenir.

Örneğin:

- Düşük performans tolere ediliyorsa, düşük performans artık standart haline gelir.
- Toksik davranış tolere ediliyorsa, toksiklik artık çalışma ortamının parçası olur.
- Güvensizlik tolere ediliyorsa, güvensizlik artık çalışma biçimine dönüşür.
- Gelişmeyen liderler tolere ediliyorsa, liderlik kalitesi organizasyonun sınırı haline gelir.
- Etik dışı davranışlar tolere ediliyorsa, etik dışılık artık yalnızca risk değil, gerçekliktir.

Bu nedenle kültür, söylenen değerlerden çok, tekrar tekrar izin verilen davranışlarla oluşur.

### 6.3 Sözler ve Gerçek Davranış Arasındaki Fark

Bir şirket kendisini saygılı, etik, kaliteli veya güven odaklı olarak tanımlayabilir. Ancak pratikte bu değerleri bozan davranışlar görmezden geliniyorsa, çalışanlar resmi söyleme değil, gerçek uygulamaya inanır.

Örneğin bir organizasyon şunu söyleyebilir:

Biz saygılı iletişime önem veririz.

Fakat bir kiři sürekli küçümseyici, sert veya toksik bir dil kullanıyor ve buna müdahale edilmiyorsa, organizasyonun gerçek mesajı şudur:

Saygısızlık burada ciddi bir problem değildir.

Benzer şekilde bir organizasyon şunu söyleyebilir:

Biz kaliteye önem veririz.

Ancak düşük kaliteli işler sürekli kabul ediliyor, aceleyle yapılan hatalı teslimatlar ödüllendiriliyor ve kalite standartları uygulanmıyorsa, gerçek kültür kalite değil, yalnızca hızlı teslim kültürüdür.

## 6.4 Kültür Posterle Değişmez

Kültür yalnızca anlatılarak, eğitim verilerek veya duvarlara değerler yazılarak değişmez. Bu araçlar bir mesaj verebilir; ancak davranış değişikliği üretmedikçe kalıcı kültür dönüşümü oluşturmaz.

Bir organizasyon şeffaflık hakkında eğitim verebilir. Fakat kararlar kapalı kapılar ardında alınıyorsa, şeffaflık kültürü oluşmaz.

Bir organizasyon öğrenme kültüründen bahsedebilir. Fakat hata yapan kişiler cezalandırılıyorsa, çalışanlar hata saklamayı öğrenir.

Bir organizasyon yüksek performans kültürünü savunabilir. Fakat düşük performansa uzun süre müdahale edilmiyorsa, yüksek performans yalnızca söylemde kalır.

Bu nedenle kültürün gerçek taşıyıcısı sözler değil, tekrar eden uygulamalardır.

## 6.5 Kültür Yavaş Değişir

Kültür hızlı tepki veren bir mekanizma değildir. Ekonomik göstergeler veya günlük operasyonel kararlar gibi ani değişmez. Kültür daha yavaş hareket eder; fakat derine işler.

İnsanlar yeni bir kültürel söylem duyduklarında hemen inanmazlar. Önce organizasyonun tutarlılığını test ederler:

- Açık konuşursam gerçekten desteklenir miyim?
- Düşük performansa gerçekten müdahale edilecek mi?
- Toksik davranış gerçekten durdurulacak mı?
- Liderler de aynı standartlara tabi olacak mı?
- Etik dışı davranışlara gerçekten sıfır tolerans gösterilecek mi?

Bu soruların cevabı sözlerle değil, zaman içinde tekrar eden kararlarla verilir.

## 6.6 K lt r  Deęiřtirmek İ in Sistem Deęiřmelidir

K lt r  deęiřtirmek i in yalnızca insanlara yeni deęerler anlatmak yeterli deęildir. Organizasyonun  alıřma bi imi yeniden tasarlanmalıdır.

Bunun i in řu alanlara m dahale etmek gerekir:

- iř akıřları,
- organizasyon yapısı,
- performans kriterleri,
- karar alma bi imleri,
- toplantı rit elleri,
-  d l ve sonu  mekanizmaları,
- liderlik davranıřları,
- geri bildirim s re leri.

   nk  k lt r, bu mekanizmaların toplam etkisiyle oluřur. Eęer sistem eski davranıřları  d llendirmeye devam ediyorsa, yeni k lt r s ylemi kalıcı hale gelemez.

## 6.7 Yeni D zene Davet Etmek Yetmez

Bir organizasyonda d n ř m isteniyorsa, insanları yeni d zene davet etmek gerekli ama yeterli deęildir. Yeni davranıřların ger ekten k lt re d n řmesi i in bu d zenin ısrarla uygulanması gerekir.

Bu s re  řu řekilde iřler:

- Yeni standart a ık a tanımlanır.
- Beklenen davranıřlar somut hale getirilir.
- Bu davranıřları g sterenler desteklenir.
- Eski davranıřlara geri d n ld ę nde m dahale edilir.
- İřtisnaların yeni norm haline gelmesine izin verilmez.
- Liderler de aynı standarda tabi olur.

K lt r deęiřimi ancak bu tutarlılık sayesinde ger ekleřir.

## 6.8 Yazılım Ekipleri Açısından Yorum

Bu ilke yazılım ekipleri için de doğrudan geçerlidir. Teknik ekiplerde kültür yalnızca mühendislik prensipleriyle değil, hangi teknik davranışların tolere edildiğiyle oluşur.

Örneğin:

- Flaky testler tolere ediliyorsa, güvensiz test kültürü oluşur.
- Code review yapılmadan merge tolere ediliyorsa, düşük kalite norm haline gelir.
- Dokümansız feature geliştirme tolere ediliyorsa, bilgi dağınıklığı kültür olur.
- Sürekli hotfix ile ilerlemek tolere ediliyorsa, plansızlık çalışma biçimine dönüşür.
- Teknik borç sürekli erteleniyorsa, karmaşıklık sistemin doğal hali haline gelir.
- Toksik senior davranışı tolere ediliyorsa, ekipte sessizlik ve korku kültürü oluşur.

Bu nedenle teknik borç yalnızca kodda bulunmaz. Bazı teknik borçlar doğrudan kültür borcudur.

## 6.9 Sonuç

Kültür, organizasyonun söylediği değerlerden çok, tekrar tekrar izin verdiği davranışların toplamıdır. Bir davranış sürekli görülüyor ve buna rağmen müdahale edilmiyorsa, o davranış artık kültürün parçası haline gelir.

Bu nedenle kültür değiştirmek isteyen bir organizasyon yalnızca yeni değerler açıklamamalıdır. Aynı zamanda iş akışlarını, performans kriterlerini, liderlik davranışlarını, karar alma biçimlerini ve tolerans sınırlarını yeniden tasarlamalıdır.

Kültür, yazılı değerler değil; tekrar tekrar izin verilen davranışların toplamıdır.



## 7 Developer Experience Kavramları

Developer Experience, kısaca geliştiricinin bir sistemi anlaması, çalıştırması, değiştirmesi, test etmesi ve güvenli şekilde üretime taşımalarının ne kadar kolay olduğunu ifade eder.

Bu konu sadece geliştiricinin rahat etmesiyle ilgili değildir. Asıl mesele şudur:

İyi Developer Experience, yazılım üretimindeki sürtünmeyi azaltır, hata ihtimalini düşürür ve geliştiricinin zihinsel yükünü hafifletir.

### 7.1 1. Daha İyi Dokümantasyon

Dokümantasyonun amacı her şeyi yazmak değildir. İyi dokümantasyonun amacı, yeni gelen bir geliştiricinin sisteme nereden başlayacağını, sistemin neden bu şekilde tasarlandığını ve bir değişikliği nasıl güvenle yapacağını anlamasını sağlamaktır.

Kötü dokümantasyon genellikle sadece komutları ve yüzeysel bilgileri verir:

Bu komutu çalıştır.  
Şu servisi başlat.  
Bu endpoint budur.

İyi dokümantasyon ise şunları da açıklar:

- Bu servis neden var?
- Hangi problemi çözüyor?
- Hangi modüllerle konuşuyor?
- Local ortamda çalıştırırken nelere dikkat edilmeli?
- En sık bozulan yerler neresi?
- Deploy sırasında riskli noktalar nelerdir?

Yani iyi dokümantasyon sadece *nasıl* sorusuna değil, aynı zamanda *neden* sorusuna da cevap verir.

### 7.2 2. Daha Kolay Local Setup

Local setup, geliştiricinin projeyi kendi bilgisayarında ne kadar kolay ayağa kaldırabildiğini ifade eder.

Kötü bir local setup örneği şudur:

README eski.  
.env eksik.  
Docker çalışmıyor.  
Database migration yarım.  
Bir servis ayrı terminalde başlatılmalı ama yazmıyor.  
Port dolu kalıyor.  
Backend ayağa kalkmış gibi görünüyor ama eski process'e istek gidiyor.

İyi bir local setup ise mümkün olduğunca sade ve tekrarlanabilir olmalıdır:

```
cp .env.example .env
docker compose up
npm install
npm run dev
```

Ama mesele sadece komutların kısa olması değildir. Asıl amaç, bu komutlar çalıştırıldığında projenin gerçekten ayağa kalkmasıdır.

Yeni gelen bir geliştirici iki gün environment kurmakla uğraşmamalı; mümkünse kısa sürede sistemi çalıştırabilir hale gelmelidir.

Kötü local setup, projeye duyulan güveni daha ilk günden zedeler.

### 7.3 3. Daha Tutarlı Tooling

Tooling, geliştiricinin kullandığı teknik araçları ifade eder. Bunlara şunlar dahildir:

- Lint araçları,
- format araçları,
- test runner,
- build sistemi,
- package manager,
- commit hook,
- CI pipeline,
- debug scriptleri,
- generate scriptleri,
- migration araçları,
- seed data araçları.

Tutarlı tooling şu anlama gelir:

Her geliştirici aynı komutları çalıştırdığında aynı sonucu almalıdır.

Kötü tooling örneği şudur:

```
npm test
```

Bu komut bir geliştiricinin bilgisayarında çalışıyor, diğerinde çalışmıyorsa tooling tutarlı değildir.

Benzer şekilde bir geliştirici **npm**, diğeri **yarn**, diğeri **pnpm** kullanıyorsa; biri Node 18, diğeri Node 22 ile çalışıyorsa sistem zamanla “bende çalışıyor” problemine düşer.

İyi tooling şunları sağlar:

- Node versiyonu bellidir.
- Package manager bellidir.
- Format otomatik çalışır.
- Lint otomatik çalışır.
- Test komutları nettir.
- Build komutu nettir.
- CI ortamı ile local ortam mümkün olduğunca aynı davranır.

Yani sistem kişisel alışkanlıklara değil, ortak standarda göre çalışır.

## 7.4 4. Secure by Default

Secure by default, sistemin varsayılan olarak güvenli davranması anlamına gelir.

Başka bir ifadeyle:

Geliştirici ekstra güvenlik düşünmese bile sistemin temel davranışı güvenli olmalıdır.

Kötü örnekler şunlardır:

```
const isAdmin = true;
```

veya:

```
if (!user) {  
  allowAccess();  
}
```

Bu tarz yaklaşımlarda güvenlik geliştiricinin dikkatine bırakılmıştır. Bu tehlikelidir.

İyi bir secure by default yaklaşımı şunları içerir:

- Varsayılan olarak erişim kapalıdır.
- Yetki açıkça verilirse erişim açılır.
- Secret bilgileri repoya girmez.

- Cookie'ler `httpOnly` olur.
- CORS gereksiz yere geniş açık bırakılmaz.
- Input validation varsayılan olarak uygulanır.
- SQL sorguları doğrudan string birleştirme ile yazılmaz.
- Authentication ve authorization merkezi middleware'lerle kontrol edilir.

Buradaki temel prensip şudur:

Güvenlik geliştiricinin hafızasına bırakılmaz; mimarinin içine gömülür.

## 7.5 5. Shifting Left

Shifting Left, yazılım geliştirme sürecinde hataları mümkün olduğunca erken yakalamak anlamına gelir.

Klasik kötü akış şöyledir:

Kod yazıldı -> deploy edildi -> kullanıcı gördü -> hata çıktı

Shift Left yaklaşımı ise şunu ister:

Kod yazarken yakala.  
Commit atmadan yakala.  
Pull request aşamasında yakala.  
CI aşamasında yakala.  
Production'a gitmeden yakala.

Hata ne kadar erken yakalanırsa, düzeltme maliyeti o kadar düşer.

Shift Left örnekleri şunlardır:

- TypeScript type checking,
- lint kontrolleri,
- unit testler,
- component testler,
- contract testler,
- security scan,
- dependency vulnerability scan,
- accessibility testleri,
- static analysis,
- pre-commit hook,

- CI pipeline.

Shift Left'in özü şudur:

Kaliteyi sürecin sonuna bırakma; kaliteyi geliştirme sürecinin içine yerleştir.

## 7.6 6. Daha Anlaşılır Modül Sınırları

Modül sınırı, sistemde hangi parçanın hangi sorumluluğa sahip olduğunu ifade eder.

Kötü modül sınırı örneği şudur:

```
Component API çağırıyor.  
Service state değiştiriyor.  
Helper auth kontrolü yapıyor.  
Reducer içinde business logic var.  
UI içinde permission logic var.
```

Bu durumda sorumluluklar birbirine karışır. Her şey her yere bulaşır. İyi modül sınırı ise daha net bir yapı kurar:

```
UI sadece gösterir.  
Facade ekranın ihtiyacını hazırlar.  
Service API ile konuşur.  
Store state tutar.  
Domain layer iş kurallarını taşır.  
Utils saf yardımcı fonksiyonlar içerir.
```

Modül sınırlarının net olması şunları sağlar:

- Bug'ın nerede olduğu daha kolay bulunur.
- Test yazmak kolaylaşır.
- AI kodu daha rahat anlar.
- Developer değişiklik yapmaktan daha az korkar.
- Bir yeri düzeltirken başka bir yeri bozma ihtimali azalır.

İyi modül sınırı, sistemin zihinsel haritasını temizler.

## 7.7 7. DDD: Domain-Driven Design

DDD, yani Domain-Driven Design, yazılımı teknik tablolara göre değil, iş alanının gerçek kavramlarına göre tasarlama yaklaşımıdır.

Basitçe:

Kodun dili, işin diliyle aynı olmalıdır.

Örneğin bir sağlık uygulamasında kötü isimlendirmeler şunlar olabilir:

```
DataItem  
RecordObject  
UserAction  
ProcessModel
```

Daha iyi isimlendirmeler ise domain kavramlarına yakın olur:

```
Patient  
Appointment  
MedicationReminder  
HealthMetric  
ConsultationSummary  
Prescription
```

DDD şu soruları sorar:

- Bu sistemin gerçek dünyadaki kavramları nelerdir?
- Bu kavramlar arasında hangi ilişkiler vardır?
- Hangi iş kuralları asla bozulmamalıdır?
- Hangi modül hangi domain alanına aittir?
- Kodda kullanılan dil ile iş tarafının kullandığı dil örtüşüyor mu?

DDD özellikle büyük sistemlerde önemlidir. Çünkü sistem büyüdükçe teknik isimler yetersiz kalır. İş kuralları kodun içine düzgün yerleşmezse proje zamanla karmaşık ve kırılgan hale gelir.

## 7.8 8. Daha Hızlı Feedback Loop'ları

Feedback loop, geliştiricinin yaptığı değişikliğin doğru mu yanlış mı olduğunu ne kadar hızlı öğrendiğini ifade eder.

Kötü feedback loop örneği şudur:

```
Kod değiştir.  
Build 8 dakika sürer.  
Testler 20 dakika sürer.  
Deploy 30 dakika sürer.  
Hata production'da fark edilir.
```

İyi feedback loop ise şöyledir:

Dosyayı kaydet.  
TypeScript hemen uyarır.  
Hot reload çalışır.  
Unit test hızlı döner.  
Component test sonucu hızlı görünür.  
CI kısa sürede feedback verir.  
Preview environment otomatik açılır.

Hızlı feedback loop geliştiriciyi hızlandırır. Fakat asıl değeri daha derindir:

Yanlış yola girdiğini erken fark etmeni sağlar.

Bu özellikle AI ile kod yazarken daha da önemlidir. Çünkü AI hızlı kod üretebilir, fakat ürettiği kodun doğru olduğu garanti değildir. Feedback loop yavaşsa, AI'nın ürettiği hatalı kod büyük bir karmaşaya dönüşebilir.

## 7.9 Bu Kavramların Ortak Amacı

Bütün bu kavramlar aynı ana amaca hizmet eder:

- Geliştiricinin zihinsel yükünü azaltmak,
- sistemi daha anlaşılır hale getirmek,
- hataları daha erken yakalamak,
- güvenliği varsayılan hale getirmek,
- yeni özellik geliştirmeyi korkutucu olmaktan çıkarmak,
- ekip içi üretimi daha tutarlı hale getirmek.

Developer Experience bu yüzden sadece konfor meselesi değildir.  
Daha doğru ifade şudur:

Developer Experience, yazılım organizasyonunun üretim kalitesini belirleyen altyapıdır.

Kötü Developer Experience olan yerde iyi mühendis bile yavaşlar.

İyi Developer Experience olan yerde ise ortalama bir mühendis bile daha güvenli, daha hızlı ve daha tutarlı üretim yapabilir.

## 7.10 Özet

| Kavram                 | Anlamı                                           |
|------------------------|--------------------------------------------------|
| Daha iyi dokümantasyon | Sistemi anlamayı kolaylaştırır.                  |
| Daha kolay local setup | Projeye giriş sürtünmesini azaltır.              |
| Tutarlı tooling        | Herkesin aynı şekilde çalışmasını sağlar.        |
| Secure by default      | Güvenliği kişisel dikkate bırakmaz.              |
| Shifting Left          | Hataları erken yakalar.                          |
| Modül sınırları        | Sistemin zihinsel haritasını temizler.           |
| DDD                    | Kodun iş dünyasını doğru temsil etmesini sağlar. |
| Feedback loop          | Yanlışları hızlı fark ettirir.                   |

## 7.11 Üst Kavram: Engineering Experience Architecture

Bu kavramların tamamı daha üst bir başlık altında düşünülebilir:

### Engineering Experience Architecture

Bu, sadece geliştiricinin rahat etmesi değil; geliştiricinin sistemi anlama, değiştirme, test etme, güvenli çalıştırma ve üretime taşıma deneyiminin bilinçli şekilde tasarlanmasıdır.

Başka bir ifadeyle:

Engineering Experience Architecture, yazılım geliştirme sürecinin insan, araç, mimari, güvenlik ve kalite boyutlarını birlikte tasarlama disiplini.

## 8 Codebase’de Niyet Aktarımı İçin Claude Template’i

### 8.1 Amaç

Bu dokümanın amacı, bir yapay zekâ asistanına yalnızca kod değişikliği yaptırmak değil, önce codebase’in taşıdığı niyeti anlamasını sağlamaktır. Burada niyet aktarımı, kodun yalnızca teknik olarak ne yaptığını değil, neden o şekilde tasarlandığını, hangi kullanıcı veya iş ihtiyacına karşılık geldiğini ve hangi davranışların korunması gerektiğini açık biçimde ortaya koyma sürecidir.

Bir codebase, yalnızca çalışan kodlardan oluşmaz. Aynı zamanda sistemin amacı, domain kavramları, kullanıcı akışları, teknik sınırları, yan etkileri ve korunması gereken davranış sözleşmeleri hakkında bilgi taşır. Bu nedenle Claude gibi bir yapay zekâ aracına doğrudan refactor yaptırmadan önce, ondan bir *Intent Map*, yani niyet haritası çıkarması istenmelidir.



## 8.2 Claude’a Verilecek Ana Prompt

You are not just a coding assistant.

Your first task is to understand the intent of this codebase before suggesting or changing anything.

I want you to analyze the codebase from the perspective of INTENT TRANSFER.

Intent transfer means:

- understanding why the code exists,
- what user/business goal it serves,
- which parts represent domain logic,
- which parts are technical implementation details,
- which boundaries must not be broken,
- which behaviors must be preserved,
- and which names or structures fail to communicate their purpose clearly.

Please do not refactor immediately.

First, produce an Intent Map of the codebase.

## 8.3 1. Codebase Amacının Analizi

Claude’dan öncelikle codebase’in temel amacını analiz etmesi istenmelidir. Bu aşamada hedef, sistemin teknik dosya yapısından önce işlevsel ve kavramsal amacını anlamaktır.

Analyze and explain:

1. What is the main purpose of this codebase?
2. What user problems does it solve?
3. What are the main business/domain concepts?
4. Which parts are core domain logic?
5. Which parts are infrastructure or technical plumbing?
6. Which parts are UI-only behavior?
7. Which parts are side effects?

Bu bölüm, yapay zekânın yalnızca dosya ve fonksiyon seviyesinde değil, sistemin genel varlık sebebi üzerinden düşünmesini sağlar.

## 8.4 2. Mimari Niyet Analizi

Bir codebase’de niyet aktarımı yalnızca isimlendirme ile sağlanmaz. Dosya yapısı, katmanlar, veri akışı ve bağımlılık yönleri de sistemin niyetini taşır. Bu nedenle Claude’dan mimari sınırları incelemesi istenmelidir.

Analyze the architecture and explain:

1. What are the main layers of the system?
2. What is the responsibility of each layer?
3. Does data flow in a clear direction?
4. Are there any circular dependencies or unclear ownership?
5. Are components/services/stores/effects/facades used with clear intent?
6. Which files currently mix too many responsibilities?
7. Which boundaries should be protected?

Bu analiz, sistemin hangi parçalarının UI, state management, API, domain logic, mapping veya infrastructure sorumluluğu taşıdığını netleştirmeye yardımcı olur.

### 8.5 3. İsimlendirme Niyeti Denetimi

İsimlendirme, niyet aktarımının en önemli araçlarından biridir. Zayıf isimler, kodun teknik olarak doğru çalışmasına rağmen anlam kaybına sebep olur. Bu nedenle Claude'dan belirsiz isimleri tespit etmesi ve daha niyet taşıyan alternatifler önermesi istenmelidir.

Review the naming across the codebase.

Find names that are too vague, such as:

- data
- item
- value
- result
- temp
- helper
- util
- handleSomething
- updateData
- setState
- processResponse
- summary
- status

For each weak name, suggest a better name that communicates intent.

Use this format:

Current name:

Suggested name:

Why this name is better:

Risk if left unchanged:

Bu bölümün amacı, kodda kullanılan teknik isimlerin domain anlamını yeterince taşıyıp taşımadığını incelemektir.

## 8.6 4. Domain Model Denetimi

Codebase’de backend DTO’ları ile frontend domain modellerinin karıştırılması, niyet aktarımını zayıflatır. Teknik olarak aynı tipte olan iki alan, domain açısından tamamen farklı anlamlara sahip olabilir.

Örneğin:

```
summary
manualSummary
generatedSummary
editedSummary
fileName
storageKey
cutNumber
internalAudioId
```

Bu yüzden Claude’dan domain modelleri, DTO’lar, interface’ler ve state type’ları üzerinde semantik analiz yapması istenmelidir.

Analyze the domain models, interfaces, DTOs, and state types.

Explain:

1. Which types represent backend DTOs?
2. Which types represent frontend/domain models?
3. Are backend names leaking into the domain layer?
4. Are there fields with the same technical type but different meaning?
5. Are there fields that should be separated semantically?

Suggest clearer domain names where needed.

Bu analiz sayesinde sistemdeki veri alanlarının yalnızca teknik şekli değil, taşıdığı anlam da görünür hale gelir.

## 8.7 5. Kullanıcı Niyeti Akışlarının Haritalanması

Bir kullanıcı eylemi, sistem içinde açık bir uygulama niyetine dönüşmelidir. Örneğin bir butona tıklamak yalnızca bir click event değildir; çoğu zaman bir iş niyetini temsil eder.

Claude’dan önemli kullanıcı akışlarını aşağıdaki formatta çıkarması istenmelidir:

Map the most important user flows.

For each flow, use this structure:

User action:  
UI event:  
Application intent:  
State/action/event:  
Side effect:  
API call:  
State update:  
Expected UI result:  
Behavior that must be preserved:

Örnek:

User action:  
User clicks Save Manual Summary

UI event:  
onSaveClicked()

Application intent:  
Save clinician-edited manual summary

State/action/event:  
manualSummarySaveRequested

Side effect:  
Send PUT request to backend

API call:  
PUT /appointment-summary/manual

State update:  
manualSummary is updated, aiSummary remains unchanged

Expected UI result:  
The edited summary remains visible after save

Behavior that must be preserved:  
Saving or reordering manual summary must not overwrite AI-generated summary

Bu yapı, kullanıcı davranışından state değişimine kadar olan zincirde niyetin kaybolmasını engeller.

## 8.8 6. Yan Etki Denetimi

Codebase'de yan etkiler, sistem davranışının en riskli alanlarından biridir. HTTP istekleri, WebSocket mesajları, localStorage kullanımı, navigation, timer, dosya indirme/yükleme ve notification gibi işlemler açıkça izole edilmelidir.

Claude'dan yan etkileri şu şekilde analiz etmesi istenmelidir:

Identify all side effects in the codebase.

Side effects include:

- HTTP requests
- WebSocket / SignalR events
- localStorage/sessionStorage
- timers
- navigation
- file upload/download
- audio/video recording
- external API calls
- notifications/toasts
- NgRx effects
- browser APIs
- clipboard actions

For each side effect, explain:

1. Where is it triggered?
2. What user or business intent does it represent?
3. Is it isolated in the correct layer?
4. Is the naming clear?
5. What can go wrong?
6. What test should protect it?

Bu bölüm, özellikle karmaşık frontend uygulamalarında davranış hatalarını azaltmak için önemlidir.

## 8.9 7. Davranış Sözleşmelerinin Çıkarılması

Davranış sözleşmesi, sistemde her koşulda doğru kalması gereken davranışları ifade eder. Bu sözleşmeler testlerle korunmalı ve refactor sırasında bozulmamalıdır.

Claude'dan davranış sözleşmelerini aşağıdaki formatta çıkarması istenmelidir:

Extract the behavior contracts of this codebase.

A behavior contract means:

"This behavior must always remain true."

Use this format:

Contract:  
Why it matters:  
Files involved:  
What could break it:  
Recommended test:

Örnek davranış sözleşmeleri:

- Manual summary edits must not overwrite AI-generated summary.
- Reordering sections must preserve section content.
- A canceled transcription state should show a temporary pause message.
- A failed token refresh should not show a user-facing toast until refresh truly fails.
- Uploaded audio files should only show audio-related references.

Bu aşama, codebase'in kritik davranışlarının açık hale gelmesini sağlar.

## 8.10 8. Test Niyeti Denetimi

Testler yalnızca teknik doğrulama araçları değildir. Aynı zamanda sistemin davranış niyetini belgeleyen sözleşmelerdir. Bu nedenle testlerin hangi davranışı koruduğu açık olmalıdır.

Claude'dan testleri veya eksik testleri şu sınıflandırma ile incelemesi istenmelidir:

Review the tests or suggest missing tests.

Classify tests by intent:

1. Unit tests
2. Component tests
3. Integration tests
4. Contract tests
5. E2E tests
6. Regression tests
7. Side-effect tests
8. State transition tests
9. Permission/authorization tests
10. Error-state tests
11. Loading-state tests
12. Retry/timeout tests

For each important behavior, suggest:

Test name:  
Test type:

What it protects:  
Why this behavior matters:

Bu yaklaşım, testlerin yalnızca coverage üretmesini değil, sistemin niyetini korumasını sağlar.

## 8.11 9. Davranış Değiştirmeden Refactor Planı

Claude doğrudan kodu değiştirmemelidir. Önce davranışı koruyan, küçük ve gözden geçirilebilir bir refactor planı üretmelidir.

After understanding the codebase, propose a refactor plan.

Do not change behavior.

Group suggestions into:

1. Rename only
2. Move/extract only
3. Type/model cleanup
4. Side-effect isolation
5. State/action cleanup
6. Test coverage improvements
7. Risky changes that should be avoided for now

For every proposed change, include:

Current problem:  
Suggested change:  
Intent improved:  
Risk level:  
Files affected:  
Suggested test:

Bu bölüm, yapay zekânın kontrolsüz refactor yapmasını engeller ve değişiklikleri daha güvenli hale getirir.

## 8.12 10. Claude'dan Beklenen Nihai Çıktı Formatı

Claude'un üretmesi gereken nihai çıktı aşağıdaki yapıda olmalıdır:

```
# Codebase Intent Map

## 1. Main Purpose
## 2. Core Domain Concepts
## 3. Architecture Layers
## 4. Data Flow
```

```
## 5. User Intent Flows
## 6. Side Effects
## 7. Weak Naming Areas
## 8. Domain Model Problems
## 9. Behavior Contracts
## 10. Missing Tests
## 11. Refactor Plan
## 12. Risks
## 13. Recommended Next Step
```

Bu çıktı, codebase'in yalnızca teknik durumunu değil, anlam haritasını da ortaya koyar.

## 8.13 Kısa Kullanım Template'i

Daha pratik kullanım için aşağıdaki kısa prompt doğrudan Claude'a verilebilir:

```
Analyze this codebase for INTENT TRANSFER.
```

```
Do not refactor immediately.
```

```
I want you to understand and document:
```

1. What this codebase is trying to achieve.
2. What the core domain concepts are.
3. Which files represent UI, state, API, side effects, domain logic, mapping, and infrastructure.
4. Whether names clearly communicate business/user intent.
5. Whether backend DTO names are leaking into domain models.
6. Whether user actions are translated into clear application intents.
7. Whether side effects are isolated and named clearly.
8. Which behaviors must be preserved.
9. Which tests should exist to protect those behaviors.
10. Which refactors would improve clarity without changing behavior.

```
Please produce:
```

```
# Codebase Intent Map

## Purpose
## Domain Concepts
## Architecture Boundaries
## User Intent Flows
## Side Effects
## Weak Names
## Domain Model Issues
## Behavior Contracts
```



```
## Missing Tests
## Safe Refactor Plan
```

```
For each weak area, explain:
- current problem
- why it hurts intent transfer
- suggested improvement
- risk level
- files affected
- test that should protect it
```

## 8.14 Refactor Aşaması İçin İkinci Prompt

İlk analiz tamamlandıktan sonra Claude'a aşağıdaki ikinci prompt verilebilir:

Now use the Intent Map you created.

Refactor the codebase to improve intent transfer, but do not change behavior.

Rules:

1. Preserve all existing behavior.
2. Prefer clearer names over comments.
3. Extract helpers only when they reveal domain intent.
4. Separate UI logic, domain logic, mapping, API calls, and side effects.
5. Do not introduce unnecessary abstraction.
6. Do not rename public API fields unless mapping is added.
7. Add or update tests for every behavior contract affected.
8. Keep changes small and reviewable.

Before writing code, show:

- files you will change
- why each file needs change
- behavior that must stay unchanged
- tests you will run or add

Then provide the full updated files.

## 8.15 Önerilen Kullanım Sırası

Claude ile codebase üzerinde çalışırken önerilen sıra şu şekilde olmalıdır:

1. Önce codebase için bir *Intent Map* çıkarılmalıdır.
2. Ardından zayıf isimlendirme alanları tespit edilmelidir.
3. Sonra davranış sözleşmeleri çıkarılmalıdır.
4. Daha sonra yan etkiler ve teknik riskler belirlenmelidir.

5. Eksik testler listelenmelidir.
6. En son, küçük ve davranış deęiřtirmeyen refactor adımları uygulanmalıdır.

## 8.16 Sonuç

Codebase’de niyet aktarımı, temiz kod yazmaktan daha geniş bir kavramdır. Temiz kod genellikle okunabilirlik ve sadelik üzerine odaklanırken, niyet aktarımı kodun arkasındaki amacı, korunması gereken davranışları ve domain anlamını da görünür hale getirir.

Bu nedenle Claude’a doğrudan “refactor et” demek yerine, önce sistemin niyetini anlaması için yapılandırılmış bir analiz yaptırmak gerekir. Doğru süreç şu şekilde özetlenebilir:

### Intent Map

- > Behavior Contracts
- > Side Effect Audit
- > Naming and Domain Model Audit
- > Missing Tests
- > Safe Refactor Plan
- > Controlled Code Changes

Bu yaklaşım sayesinde codebase yalnızca makine tarafından çalıştırılabilir bir yapı olmaktan çıkar; insan ve yapay zekâ tarafından doğru anlaşılabilir bir anlam mimarisine dönüşür.

## 9 Codebase’de Niyet Aktarımı ile İlişkili Sektörel Kavramlar

### 9.1 Giriş

Yazılım geliştirme pratiğinde “niyet aktarımı” ifadesi tek başına standartlaşmış bir endüstri terimi değildir. Ancak bu kavram, yazılım mimarisi, temiz kod, domain modelleme, test stratejisi ve teknik dokümantasyon alanlarında kullanılan çeşitli kavramların birleşimi olarak değerlendirilebilir.

Bu bağlamda niyet aktarımı; bir codebase’in yalnızca ne yaptığını değil, neden o şekilde tasarlandığını, hangi kullanıcı veya iş amacına hizmet ettiğini, hangi davranışları koruması gerektiğini ve hangi teknik sınırlar içinde çalıştığını açık biçimde ifade etmesi anlamına gelir.

Bu dokümanda, codebase’de niyet aktarımıyla doğrudan ilişkili temel kavramlar açıklanmaktadır.

### 9.2 1. Intention-Revealing Interface

*Intention-Revealing Interface*, bir fonksiyonun, sınıfın, metodun, component’in veya API’nin isminin ve kullanım biçiminin, onun teknik işleminden ziyade kullanım niyetini açıkça göstermesi anlamına gelir.

Bu kavramın temel amacı, kodu okuyan kişinin şu soruya hızlı cevap verebilmesidir:

Bu yapı teknik olarak ne yapıyor değil, hangi amaçla kullanılıyor?

Örneğin aşağıdaki isim teknik olarak bir işlem yapıldığını gösterir; ancak niyetini yeterince açıklamaz:

```
updateData()
```

Buna karşılık aşağıdaki isim, yapılan işlemin domain bağlamındaki amacını daha açık ifade eder:

```
saveManualSummary()
```

Daha da açık bir davranış sözleşmesi gerektiğinde test ismi şu şekilde yazılabilir:

```
shouldSaveManualSummaryWithoutOverwritingGeneratedSummary()
```

Bu yaklaşımda önemli olan, fonksiyon isimlerini gereksiz biçimde uzatmak değil, isimlerin domain niyetini taşımasını sağlamaktır. Bir metodun adı, onun hangi kullanıcı eylemini veya iş kuralını temsil ettiğini göstermelidir.

## 9.3 2. Ubiquitous Language

*Ubiquitous Language*, Domain-Driven Design yaklaşımında kullanılan temel kavramlardan biridir. Anlamı, ürün sahipleri, domain uzmanları, yazılımcılar, tester'lar ve kodun kendisinin aynı kavramsal dili kullanmasıdır.

Bu kavram, yazılım sistemlerinde anlam kaybını azaltmak için önemlidir. Çünkü bir kavram farklı ekip üyeleri tarafından farklı anlamlarda kullanılıyorsa, bu durum hem kodda hem testlerde hem de iş kararlarında hatalara yol açabilir.

Örneğin aşağıdaki kavramlar teknik olarak birbirine benzeyebilir:

```
summary  
manualSummary  
generatedSummary  
editedSummary  
clinicianEditedSummary
```

Fakat domain açısından her biri farklı anlam taşıyabilir. Eğer sistemde hem yapay zekâ tarafından üretilmiş bir özet hem de kullanıcı tarafından düzenlenmiş manuel bir özet varsa, bu iki alanın aynı isim altında toplanması niyet aktarımını zayıflatır.

Bu nedenle *Ubiquitous Language*, codebase'in iş domain'iyle aynı dili konuşmasını sağlar. Kodda kullanılan isimler, ekip içinde kullanılan iş terimleriyle uyumlu olmalıdır.

### 9.4 3. Domain-Driven Design

*Domain-Driven Design*, yazılım sisteminin teknik detaylardan önce iş domain'i etrafında modellenmesini savunan bir yazılım tasarım yaklaşımıdır.

Bu yaklaşımda temel amaç, sistemin merkezine teknik veri yapılarından ziyade gerçek iş kavramlarını yerleştirmektir. Sistem; kullanıcı davranışları, iş kuralları, domain nesneleri ve bağlamsal sınırlar üzerinden tasarlanır.

Domain-Driven Design, niyet aktarımı açısından önemlidir çünkü codebase'in yalnızca teknik bir yapı değil, domain bilgisini taşıyan bir model olmasını sağlar.

Örneğin bir sistemde aşağıdaki alanlar bulunabilir:

```
cutNumber
internalAudioId
storageKey
libloc
libname
systemId
```

Bu alanlar ilk bakışta teknik veri alanları gibi görünebilir. Ancak domain içinde her birinin ayrı bir anlamı vardır. Örneğin `cutNumber` kullanıcıya veya dış sisteme ait bir ses kaydı numarasını temsil ederken, `internalAudioId` sistemin kendi içinde kullandığı benzersiz kimliği temsil edebilir.

Bu ayrım korunmazsa sistemin niyeti bozulur. Domain-Driven Design, bu tür ayrımları açık hale getirir ve kodun iş anlamını korumasına yardımcı olur.

### 9.5 4. Bounded Context

*Bounded Context*, Domain-Driven Design içinde kullanılan önemli bir kavramdır. Bir domain kavramının yalnızca belirli bir bağlam içinde kesin anlam taşıdığını ifade eder.

Aynı kelime, farklı bağlamlarda farklı anlamlara gelebilir. Bu nedenle büyük sistemlerde tek bir evrensel model yerine, bağlama göre ayrılmış modeller kullanılır.

Örneğin `status` kelimesi farklı bağlamlarda farklı anlamlara sahip olabilir:

```
appointmentStatus
transcriptionStatus
uploadStatus
paymentStatus
connectionStatus
```

Bunların hepsine yalnızca `status` demek, kodun niyetini zayıflatır. Çünkü okuyan kişi bu statünün hangi bağlama ait olduğunu anlamakta zorlanır.

*Bounded Context* yaklaşımı, bu belirsizliği azaltır. Her kavramın hangi bağlamda geçerli olduğunu belirler. Böylece domain dili daha tutarlı hale gelir.

## 9.6 5. Self-Documenting Code

*Self-Documenting Code*, kodun kendi yapısı, isimlendirmesi ve organizasyonu sayesinde ayrıca uzun açıklamalara ihtiyaç duymadan anlaşılabilir olmasıdır.

Bu kavram genellikle temiz kod yaklaşımıyla ilişkilendirilir. Amaç, yorum satırlarına aşırı bağımlı olmayan, isimleri ve yapısı itibarıyla anlam taşıyan kod yazmaktır.

Örneğin aşağıdaki kodda niyet zayıftır:

```
if (status === 'Canceled') {  
    showMessage();  
}
```

Daha niyet açıklayıcı bir yapı şu şekilde olabilir:

```
if (isTranscriptionTemporarilyPaused(status)) {  
    showTranscriptionPauseMessage();  
}
```

İkinci örnekte teknik durum, domain anlamına çevrilmiştir. Artık kodu okuyan kişi **Canceled** değerinin teknik bir enum değeri olduğunu değil, sistem açısından geçici bir transcription pause durumunu temsil ettiğini görür.

Ancak *Self-Documenting Code*, niyet aktarımının tamamını karşılamaz. Çünkü bazı kararların nedeni yalnızca koddan anlaşılamaz. Bu tür durumlarda ek dokümantasyon veya mimari karar kayıtları gerekir.

## 9.7 6. Architecture Decision Record

*Architecture Decision Record*, kısa adıyla *ADR*, önemli mimari kararların neden alındığını belgeleyen dokümandır.

Bir codebase'de bazı kararlar koddan görülebilir; ancak neden o kararın alındığı her zaman açık değildir. ADR, bu boşluğu doldurur.

Bir ADR genellikle şu bilgileri içerir:

- Kararın başlığı,
- Bağlam,
- Alınan karar,
- Kararın gerekçesi,
- Alternatifler,
- Sonuçlar ve etkiler.

Örneğin bir sistemde ses dosyalarının S3 üzerinde `cutNumber` yerine `internalAudioId` ile saklanmasına karar verilmiş olabilir. Bu kararın teknik nedeni, `cutNumber` değerinin global olarak benzersiz olmaması olabilir.

Bu durumda ADR şu niyeti korur:

Decision:

Use `internalAudioId` as the S3 object key.

Reason:

`cutNumber` is not globally unique across libraries.

Consequence:

Audio lookup must first resolve `systemId`, `libloc`, `libname`, and `cutNumber` into `internalAudioId`.

Bu tür kayıtlar, gelecekteki geliştiricilerin veya yapay zekâ araçlarının sistemi yanlış yorumlamasını engeller.

## 9.8 7. Behavior Contract

*Behavior Contract*, sistemde her koşulda korunması gereken davranışları ifade eder. Bu kavram, bir iş kuralının veya kritik sistem davranışının açık bir sözleşme olarak tanımlanmasıdır.

Bir behavior contract şu cümleyle özetlenebilir:

Bu davranış her zaman doğru kalmalıdır.

Örnek:

Manual summary edits must not overwrite AI-generated summary.

Bu ifade yalnızca bir test senaryosu değildir. Aynı zamanda sistemin domain açısından önemli bir kuralıdır. Kod refactor edilse bile bu davranış bozulmamalıdır.

Behavior contract'lar genellikle şu araçlarla korunur:

- Unit testler,
- Integration testler,
- Contract testler,
- E2E testler,
- Acceptance criteria,
- Regression testler.

Niyet aktarımı açısından behavior contract'lar çok önemlidir. Çünkü bir codebase'in anlamı yalnızca veri modellerinde değil, korunması gereken davranışlarda da saklıdır.

## 9.9 8. Invariant

*Invariant*, bir sistemin veya modelin yaşam döngüsü boyunca her zaman doğru kalması gereken koşuldur.

Behavior contract ile yakından ilişkilidir; fakat daha çok modelin iç tutarlılığına odaklanır.

Örneğin:

`A paid invoice cannot have a negative total amount.`

veya:

`An uploaded audio file must always be associated with a valid library.`

Invariant'lar sistemin bozulmaması gereken temel doğrularını ifade eder. Eğer bir invariant ihlal edilirse, sistem teknik olarak çalışmaya devam etse bile domain açısından geçersiz bir duruma düşer.

Codebase'de niyet aktarımı yapılırken invariant'ların açıkça belirtilmesi gerekir. Çünkü bu kurallar çoğu zaman kodun içine dağılmış durumda olur ve yeni bir geliştirici tarafından kolayca fark edilmeyebilir.

## 9.10 9. Business Rule

*Business Rule*, sistemin iş domain'inden gelen kuraldır. Teknik bir zorunluluktan değil, ürünün veya kurumun çalışma biçiminden kaynaklanır.

Örneğin:

`External users must not see unflagged audio cuts.`

veya:

`A user can download an audio file only after the file exists in storage.`

Business rule'lar kodun en önemli niyet kaynaklarından biridir. Çünkü sistemin neden belirli bir şekilde davrandığını açıklarlar.

Bu kurallar kodda belirsiz isimlerle, dağınık condition'larla veya yorum satırlarına gizlenmiş şekilde duruyorsa, niyet aktarımı zayıflar. İyi bir codebase'de business rule'lar açık fonksiyonlar, testler, domain servisleri veya policy nesneleriyle temsil edilmelidir.

Örneğin:

`canExternalUserAccessCut(user, cut)`

Bu isim, teknik bir kontrol değil, doğrudan iş kuralı niyeti taşır.

## 9.11 10. Acceptance Criteria

*Acceptance Criteria*, bir feature'ın veya değişikliğin kabul edilebilir sayılması için sağlaması gereken koşullardır.

Acceptance criteria genellikle ürün, iş analizi, QA ve geliştirme süreçlerinde kullanılır. Bir user story'nin tamamlanmış kabul edilmesi için hangi davranışların gerçekleşmesi gerektiğini açıklar.

Örneğin:

```
Given the user edits a manual summary,  
When the user saves the changes,  
Then the manual summary should be updated,  
And the AI-generated summary should remain unchanged.
```

Bu yapı, davranışın kullanıcı açısından nasıl doğrulanacağını gösterir. Niyet aktarımı açısından acceptance criteria önemlidir çünkü teknik implementasyondan önce beklenen davranış açıklar.

Acceptance criteria, test senaryolarına dönüştürülebilir. Böylece iş niyeti ile testler arasında doğrudan bağlantı kurulur.

## 9.12 11. Executable Specification

*Executable Specification*, sistemin beklenen davranışını hem insan tarafından okunabilir hem de otomatik olarak çalıştırılabilir testler şeklinde ifade etme yaklaşımıdır.

Bu kavram genellikle Behavior-Driven Development ile ilişkilidir. Amaç, gereksinimleri yalnızca dokümanda tutmak değil, çalışan testler haline getirmektir.

Örneğin:

Scenario: Saving manual summary preserves generated summary

```
Given an appointment has an AI-generated summary  
And the clinician edits the manual summary  
When the clinician saves the manual summary  
Then the manual summary should be updated  
And the AI-generated summary should remain unchanged
```

Bu yapı, hem iş tarafı hem teknik taraf için anlaşılardır. Aynı zamanda test olarak çalıştırılabilir hale getirildiğinde, sistemin niyetini otomatik olarak korur.

Niyet aktarımı açısından executable specification güçlüdür çünkü kodun ne yapması gerektiğini yalnızca anlatmaz; aynı zamanda doğrular.

## 9.13 12. Regression Test

*Regression Test*, daha önce doğru çalışan bir davranışın sonraki değişikliklerle bozulmadığını doğrulayan test türüdür.



Niyet aktarımı açısından regression testler özellikle önemlidir. Çünkü bir codebase'in geçmişte yaşadığı bug'lar, sistemin hangi davranışlarının hassas olduğunu gösterir.

Örneğin daha önce manuel özet kaydedilirken yapay zekâ özeti yanlışlıkla silindiyse, şu test bir regression test olur:

```
shouldNotOverwriteGeneratedSummaryWhenManualSummaryIsSaved()
```

Bu test yalnızca teknik bir doğrulama değildir. Aynı zamanda geçmişte öğrenilmiş bir domain bilgisini codebase'e kaydeder.

Bu nedenle regression testler, codebase'in hafızası gibi düşünülebilir.

## 9.14 13. Design by Contract

*Design by Contract*, yazılım bileşenlerinin birbirleriyle açık sözleşmeler üzerinden çalışmasını savunan bir yaklaşımdır.

Bu yaklaşımda bir fonksiyonun veya modülün beklenen girdileri, çıktıları ve çalışma koşulları açıkça tanımlanır.

Temel kavramlar şunlardır:

- **Precondition:** Fonksiyon çalışmadan önce doğru olması gereken koşul.
- **Postcondition:** Fonksiyon çalıştıktan sonra doğru olması gereken koşul.
- **Invariant:** Süreç boyunca her zaman doğru kalması gereken koşul.

Örneğin:

**Precondition:**  
`manualSummary must not be null.`

**Postcondition:**  
`manualSummary is updated.`

**Invariant:**  
`generatedSummary remains unchanged.`

Bu yaklaşım, niyet aktarımı açısından değerlidir çünkü bir fonksiyonun yalnızca ne yaptığını değil, hangi koşullar altında doğru kabul edildiğini de belirtir.

## 9.15 14. Semantic Naming

*Semantic Naming*, değişken, fonksiyon, sınıf ve dosya isimlerinin teknik şekilden ziyade anlamı ifade edecek biçimde seçilmesidir.

Örneğin:

```
data
result
payload
response
```

gibi isimler çoğu zaman teknik taşıyıcı isimlerdir. Fakat domain anlamı taşımazlar.

Buna karşılık:

```
appointmentSummary  
clinicianEditedSummary  
audioStorageKey  
pendingAudioDownload  
externalUserAccessPolicy
```

gibi isimler, nesnenin sistem içindeki anlamını daha açık taşır.

Semantic naming, niyet aktarımının en pratik araçlarından biridir. Kodun okunabilirliği ve bakım kolaylığı doğrudan isimlerin anlam kalitesine bağlıdır.

## 9.16 15. Information Architecture

*Information Architecture*, bilginin sistem içinde nasıl organize edildiğini ifade eder. Yazılım codebase'i bağlamında bu kavram; dosyaların, klasörlerin, modüllerin, modellerin ve akışların anlamlı bir yapı içinde düzenlenmesini kapsar.

Bir codebase'de bilgi mimarisi zayıfsa, geliştirici veya yapay zekâ sistemi anlamakta zorlanır. Örneğin bütün dosyaların **utils**, **helpers** veya **components** altında toplanması, sistemin gerçek sorumluluklarını gizler.

Daha niyet taşıyan bir yapı şu şekilde olabilir:

```
appointment-summary/  
  components/  
  containers/  
  services/  
  mappers/  
  store/  
  models/  
  tests/
```

Bu yapı, sistemin hangi parçasının hangi sorumluluğu taşıdığını açıkça gösterir. Böylece codebase'in geometrisi, sistemin niyetini destekler.

## 9.17 16. Architectural Boundary

*Architectural Boundary*, sistemdeki farklı sorumluluk alanları arasında çizilen mimari sınırdır.

Örneğin bir frontend uygulamasında şu sınırlar bulunabilir:

UI Layer  
Application State Layer  
Domain Logic Layer  
API Client Layer  
Infrastructure Layer

Bu sınırlar korunmadığında niyet aktarımı zayıflar. Örneğin bir UI component'i doğrudan HTTP çağrısı yapıyor, response mapping gerçekleştiriyor, localStorage kullanıyor ve state değiştiriyorsa, o component'in niyeti belirsiz hale gelir.

İyi bir mimari sınır, şu soruları netleştirir:

- Bu kod hangi katmana aittir?
- Hangi sorumluluğu taşır?
- Hangi detayları bilmemelidir?
- Hangi davranışı dışarıya sunar?

Architectural boundary, codebase'in niyetini katmanlar üzerinden görünür kılar.

## 9.18 17. Separation of Concerns

*Separation of Concerns*, farklı sorumlulukların birbirinden ayrılması prensibidir.

Bu prensibe göre bir modül veya dosya, mümkün olduğunca tek bir ana sorumluluğa sahip olmalıdır. UI render etmek, API çağrısı yapmak, veri dönüştürmek, validation yapmak ve state güncellemek aynı yerde toplanmamalıdır.

Örneğin zayıf bir component şu sorumlulukları aynı anda taşıyabilir:

- Form verisini okuma,
- API request hazırlama,
- HTTP çağrısı yapma,
- Response'u dönüştürme,
- Store güncelleme,
- Toast gösterme.

Daha iyi bir yapıda bu sorumluluklar ayrılır:

Component -> user interaction  
Facade -> application intent  
Effect -> side effect  
Service -> API communication  
Mapper -> DTO/domain conversion  
Reducer -> state update

Bu ayırım, kodun niyetini daha okunabilir hale getirir.

## 9.19 18. Codebase Intent Map

*Codebase Intent Map*, bir codebase'in amacını, domain kavramlarını, kullanıcı akışlarını, davranış sözleşmelerini, yan etkilerini, mimari sınırlarını ve riskli alanlarını haritalayan analiz dokümanıdır.

Bu terim her şirkette standart olarak kullanılmayabilir; ancak AI destekli codebase analizi için oldukça kullanışlı bir kavramsal çerçevedir.

Bir Codebase Intent Map genellikle şu başlıkları içerir:

- Ana amaç,
- Domain kavramları,
- Mimari katmanlar,
- Kullanıcı niyeti akışları,
- Yan etkiler,
- Zayıf isimlendirmeler,
- DTO/domain model problemleri,
- Davranış sözleşmeleri,
- Eksik testler,
- Güvenli refactor planı.

Bu harita, yapay zekâya veya yeni geliştiriciye codebase'in yalnızca dosya yapısını değil, anlam yapısını da aktarır.

## 9.20 19. Intent-Preserving Architecture

*Intent-Preserving Architecture*, sistemin mimarisinin yalnızca teknik çalışabilirliği değil, domain niyetini de koruyacak şekilde tasarlanmasıdır.

Bu yaklaşımda mimari kararlar şu soruya göre değerlendirilir:

Bu yapı sistemin gerçek amacını ve kritik davranışlarını koruyor mu?

Örneğin bir refactor sonrasında kod daha kısa hale gelebilir; fakat `manualSummary` ve `generatedSummary` ayrımı bulanıklaşıyorsa, mimari niyeti koruyamamış demektir.

Bu nedenle intent-preserving architecture, yalnızca sadelik veya performans değil, anlamın korunmasını da hedefler.

## 9.21 20. Intent-Revealing Codebase Design

*Intent-Revealing Codebase Design*, bir codebase'in tamamının niyeti açığa çıkaracak şekilde tasarlanmasıdır.

Bu kavram, tek bir fonksiyon veya sınıf seviyesindeki *Intention-Revealing Interface* fikrini daha geniş ölçekte codebase düzeyine taşır.

Bu yaklaşımda şu unsurlar birlikte değerlendirilir:

- Anlamlı isimlendirme,
- Domain dili,
- Mimari sınırlar,
- Açık kullanıcı akışları,
- İzole yan etkiler,
- Davranış sözleşmeleri,
- Testlerle korunmuş iş kuralları,
- Karar kayıtları.

Bu nedenle senin “niyet aktarımı” dediğin şeyin en uygun genel adı şu şekilde ifade edilebilir:

### Intent-Revealing Codebase Design

Türkçe karşılığı:

### Niyeti Açığa Çıkaran Codebase Tasarımı

## 9.22 Özet

Codebase’de niyet aktarımı tek bir kavrama indirgenemez. Bu kavram, birçok yazılım mühendisliği pratiğinin birleşimiyle oluşur.

| Kavram                           | Niyet Aktarımıyla İlişkisi                                               |
|----------------------------------|--------------------------------------------------------------------------|
| Intention-Revealing Interface    | Fonksiyon, sınıf ve API isimlerinin kullanım niyetini göstermesi.        |
| Ubiquitous Language              | Kodun, ekip ve domain uzmanlarıyla aynı dili konuşması.                  |
| Domain-Driven Design             | Sistemin teknik detaylardan önce domain kavramlarıyla modellenmesi.      |
| Bounded Context                  | Kavramların bağlama göre doğru anlamda kullanılması sağlanması.          |
| Self-Documenting Code            | Kodun isimleri ve yapısıyla kendini açıklaması.                          |
| Architecture Decision Record     | Mimari kararların neden alındığını belgelemesi.                          |
| Behavior Contract                | Korunması gereken davranışların açık sözleşme haline getirilmesi.        |
| Invariant                        | Her zaman doğru kalması gereken sistem koşullarını belirtmesi.           |
| Business Rule                    | İş domain’inden gelen kuralların kodda açık temsil edilmesi.             |
| Acceptance Criteria              | Beklenen kullanıcı davranışlarının kabul koşullarıyla açıklanması.       |
| Executable Specification         | İş gereksinimlerinin çalıştırılabilir testlere dönüşmesi.                |
| Regression Test                  | Geçmişte öğrenilmiş hassas davranışların korunması.                      |
| Design by Contract               | Bileşenlerin açık koşullar ve sözleşmelerle çalışması.                   |
| Semantic Naming                  | İsimlerin teknik biçimden çok anlamı taşıması.                           |
| Information Architecture         | Codebase içindeki bilginin anlamlı biçimde organize edilmesi.            |
| Architectural Boundary           | Sorumluluk alanlarının net sınırlarla ayrılması.                         |
| Separation of Concerns           | Farklı sorumlulukların birbirine karıştırılmaması.                       |
| Codebase Intent Map              | Codebase’in amaç, davranış, domain ve risk haritasının çıkarılması.      |
| Intent-Preserving Architecture   | Mimari yapının sistem niyetini ve kritik davranışları koruması.          |
| Intent-Revealing Codebase Design | Codebase’in bütünsel olarak niyeti açığa çıkaracak şekilde tasarlanması. |

Sonuç olarak, “niyet aktarımı” kavramı yazılım sektöründe en çok şu kavramlarla karşılaşılır:

Intention-Revealing Interface  
Ubiquitous Language

Domain-Driven Design  
Behavior Contracts  
Self-Documenting Code  
Architecture Decision Records

Ancak codebase'in tamamı için daha kapsayıcı bir ifade kullanılacaksa, en uygun adlandırma şudur:

### **Intent-Revealing Codebase Design**

Bu yaklaşım, kodun yalnızca çalışmasını değil, doğru anlamı taşımasını, korunması gereken davranışları görünür kılmasını ve hem insanlar hem de yapay zekâ araçları tarafından doğru yorumlanmasını hedefler.

## 10 Yazılımda Takip Edilebilirlik Geometrisi

### 10.1 Giriş

Yazılım projelerinde temel mesele her zaman her şeyi bütünüyle anlamak değildir. Modern yazılım sistemleri çoğu zaman fazla büyüktür, fazla katmanlıdır ve tarihsel olarak fazla karar biriktirmiştir. Bu nedenle bir geliştiricinin veya AI agent'ın bütün sistemi zihninde eksiksiz taşıması pratik olarak mümkün değildir.

Bu noktada asıl önemli olan şey şudur:

Bir sistemi tamamen bilmekten çok, sistem içinde doğru izi takip edebilmek gerekir.

Yani iyi bir yazılım projesi, herkesin her şeyi bildiği proje değildir. İyi bir yazılım projesi, bir değişiklik yapıldığında bunun nereden geldiğinin, nereye gittiğinin ve neyi etkilediğinin takip edilebildiği projedir.

### 10.2 Anlamak ve Takip Edebilmek Arasındaki Fark

Bir sistemi anlamak, o sistemin ne yaptığını kavramaktır. Fakat takip edebilmek, daha operasyonel ve daha güçlü bir beceridir. Takip edilebilirlik; bir feature'ın hangi spec'e bağlı olduğunu, bir kod parçasının hangi davranışı ürettiğini, bir bug'ın hangi kök nedenden kaynaklandığını ve bir değişikliğin hangi testlerle doğrulanacağını görebilmektir.

Kötü bir sistemde durum genellikle şöyledir:

Bir şey değişir.  
Neyi etkilediği belli değildir.  
Kim kullanıyor belli değildir.  
Hangi test doğruluyor belli değildir.  
Kararın neden alındığı belli değildir.  
Dokümantasyon güncel mi belli değildir.

İyi bir sistemde ise durum şöyledir:

Bir şey değişir.  
Hangi modülü etkilediği bellidir.  
Hangi kullanıcı davranışına bağlı olduğu bellidir.  
Hangi testlerle doğrulanacağı bellidir.  
Hangi API contract'a temas ettiği bellidir.  
Hangi mimari kararın sonucu olduğu bellidir.

Bu yüzden sürdürülebilir yazılım geliştirme yalnızca kod yazma veya kodu anlama meselesi değildir. Aynı zamanda iz sürebilme meselesidir.



### 10.3 Agent Açısından Takip Edilebilirlik

AI agent'lar da insan geliştiriciler gibi sınırsız bir anlayışa sahip değildir. Agent'ın başarılı olması, çoğu zaman sistemin ona ne kadar iyi iz bıraktığına bağlıdır.

Kötü projede agent tahmin eder. İyi projede agent iz sürer.

Bu cümle şu anlama gelir:

Eğer proje yapısı belirsizse, dokümantasyon eksikse, testler zayıfsa ve dosya isimleri niyeti anlatmıyorsa, agent doğru değişikliği tahmin etmeye çalışır. Fakat proje iyi yapılandırılmışsa, agent doğru dosyaya, doğru teste, doğru spec'e ve doğru davranışa iz sürerek ulaşabilir.

Bu ayrım çok kritiktir. Çünkü tahmin eden agent risklidir; iz süren agent daha güvenlidir.

### 10.4 Kötü Projede Agent'ın Davranışı

Kötü yapılandırılmış bir projede agent genellikle şu şekilde çalışır:

```
Belirsiz prompt
|
v
Dağınık codebase
|
v
Eksik dokümantasyon
|
v
Zayıf test sinyali
|
v
Tahmine dayalı değişiklik
|
v
Beklenmeyen yan etki riski
```

Bu durumda agent, sistemin gerçek niyetini koddan net biçimde okuyamaz. Hangi dosyanın sorumlu olduğunu, hangi davranışın contract olduğunu, hangi testin doğrulama sağladığını veya hangi edge case'in önemli olduğunu bilemeyebilir.

Sonuç olarak agent hızlı kod yazabilir; fakat bu hız, yanlış değişikliklerin de hızlı üretilmesi anlamına gelebilir.

## 10.5 İyi Projede Agent’ın Davranışı

İyi yapılandırılmış bir projede agent’ın davranışı daha farklıdır:

```
Açık prompt
|
v
README / CLAUDE.md / Spec
|
v
Modül haritası
|
v
İlgili dosyalar
|
v
Testler ve contract’lar
|
v
Kontrollü değişiklik
|
v
Doğrulanabilir sonuç
```

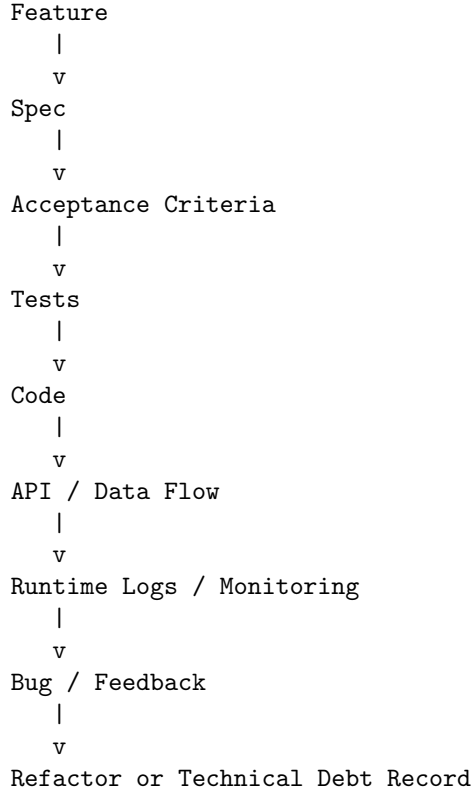
Bu durumda agent yalnızca tahmin etmez. Sistemdeki izleri takip eder. Önce dokümantasyona bakar, sonra ilgili modülleri bulur, sonra testleri inceler, sonra değişiklik yapar ve sonucu doğrular.

Bu nedenle iyi projede agent’ın görevi yalnızca kod üretmek değildir. Agent, sistemin bıraktığı izler üzerinde hareket eden bir yardımcıya dönüşür.

## 10.6 Takip Edilebilirlik Geometrisi

Takip edilebilirlik geometrisi, sistemdeki bilgi ve davranış izlerinin nasıl bağlandığını ifade eder. Bir feature, spec, test, kod, API, log, bug ve mimari karar arasında bağlantı kurulabiliyorsa sistemin takip edilebilirlik geometrisi sağlıklıdır.

Bu yapı şöyle düşünülebilir:



Bu zincir koparsa takip edilebilirlik azalır. Örneğin test varsa ama hangi spec'i doğruladığı belli değilse, testin değeri düşer. Kod varsa ama hangi kullanıcı davranışını karşıladığı belli değilse, bakım zorlaşır. Bug varsa ama root cause'a bağlanamıyorsa, aynı problem tekrar eder.

## 10.7 Takip Edilebilirlik Borcu

Takip edilebilirlik eksikliği bir teknik borç türüdür. Bu borç, sistemin anlaşılmasını, değiştirilmesini, test edilmesini ve güvenle evrilmesini zorlaştırır.

Bu borç şu şekillerde ortaya çıkar:

- Bir feature'ın hangi spec'e bağlı olduğu belli değildir.
- Bir kod değişikliğinin hangi kullanıcı davranışını etkilediği görülemez.
- API endpoint'lerinin hangi ekranlar veya servisler tarafından kullanıldığı takip edilemez.
- Bug raporundan root cause'a gidilemez.
- Root cause'dan test veya refactor planına bağlantı kurulamaz.
- Mimari kararların neden alındığı kayıtlı değildir.
- Modüller arası bağımlılıklar görünür değildir.
- Data flow diagram veya dokümanla izlenemez.
- Log, monitoring ve test çıktıları problemi takip etmeye yardım etmez.
- AI agent veya yeni developer projede iz sürerek doğru dosyaya ulaşamaz.

Bu durumda sistem yalnızca karmaşık hale gelmez; aynı zamanda güvenilmez hale gelir.

## 10.8 Yazılımda Üç Seviye

Yazılımda profesyonel çalışma üç seviyede düşünülebilir:

1. **Anlamak:** Bu kod ne yapıyor?
2. **Takip etmek:** Bu kod nereden geliyor, nereye gidiyor ve neyi etkiliyor?
3. **Kontrol etmek:** Bu değişikliği güvenle yapabilir miyim?

Birinci seviye tek başına yeterli değildir. Kodun ne yaptığını anlamak önemlidir; fakat daha kritik olan, değişikliğin etkisini takip edebilmek ve güvenle kontrol edebilmektir.

Bu yüzden iyi mühendislik yalnızca açıklama üretmez. İz üretir.

## 10.9 Sonuç

Modern yazılım sistemlerinde herkesin her şeyi bilmesi mümkün değildir. Bu nedenle sürdürülebilirlik, mutlak bilgiye değil, takip edilebilirliğe dayanır.

Bir sistemde feature'dan spec'e, spec'ten teste, testten koda, koddan API'ye, API'den runtime davranışına ve oradan teknik borç kaydına iz sürülebiliyorsa, o sistem daha güvenli biçimde geliştirilebilir.

Kötü projede agent tahmin eder. İyi projede agent iz sürer.

Bu ilke yalnızca AI agent'lar için değil, insan geliştiriciler için de geçerlidir. Çünkü iyi yazılım mimarisi, insanın ve agent'ın sistemi ezberlemesini değil, sistem içinde doğru yolu bulmasını sağlar.

## 11 Yazılım Mühendisliğinde Basitlik, Kavram-sallaştırma ve Sistem Tasarımı

### 11.1 Giriş

Yazılım mühendisliği yalnızca kod yazma faaliyeti değildir. Yazılım geliştirme; analiz, tasarım, test, bakım, planlama, öğrenme, soyut karmaşıklığı yönetme ve doğru kavramlarla sistem inşa etme sürecidir.

Bu bağlamda iyi yazılım, yalnızca çalışan koddan ibaret değildir. İyi yazılım; doğru anlaşılmış bir problem, doğru kurulmuş kavramsal model, sade tasarım, testlerle korunan davranışlar ve sürdürülebilir bir mimari üzerine kurulur.

Yazılım mühendisliği, soyut sistemleri disiplinli ama yaratıcı biçimde inşa etme sanatıdır.

### 11.2 Yazılım Mühendisliği: Mühendislik ve Zanaatkârlık Arasında

Yazılım mühendisliği, geleneksel mühendislik dalları gibi analiz, tasarım, uygulama, test, bakım, kalite, güvenlik ve sürdürülebilirlik gibi sistematik süreçler içerir. Bu yönüyle yazılım geliştirme bir mühendislik faaliyetidir.

Ancak yazılım, fiziksel dünyada değil soyut bir alanda inşa edilir. İnşaat veya makine mühendisliğinde doğa yasaları ve fiziksel sınırlar tasarımı belirlerken, yazılımda kuralların ve sınırların büyük kısmını geliştiriciler tanımlar. Bu durum yazılıma büyük bir esneklik kazandırır; fakat aynı zamanda karmaşıklığın görünmez hale gelmesine yol açar.

Bir binadaki çatlak fiziksel olarak görülebilir. Fakat yazılımdaki kötü bağımlılık, yanlış soyutlama, dağınık domain modeli veya gereksiz karmaşıklık çoğu zaman hemen fark edilmez.

Bu nedenle yazılım geliştirme hem mühendisliktir hem de zanaatkârlıktır:

- **Mikro ölçekte** temiz, okunabilir ve iyi yapılandırılmış kod yazmak zanaatkârlık ister.
- **Makro ölçekte** doğru sistemi kurmak, kaliteyi korumak ve sürdürülebilirliği sağlamak mühendislik disiplini ister.

Yazılım geliştirme, mühendislik disipliniyle yapılan bir zanaatkârlıktır.

### 11.3 Kavramsallaştırmanın Önemi

İyi yazılım, koddan önce doğru kavramlarla başlar. Bir yazılım problemi çözülmeden önce, o problemi hangi kavramlarla düşüneceğimiz belirlenmelidir.

Kavramsallaştırma şu sorulara cevap arar:

- Sistemde gerçekten hangi kavramlar var?
- Bu kavramların sorumlulukları nelerdir?

- Bu kavramlar birbirleriyle nasıl ilişki kurar?
- Sistemin sınırı nerede başlar ve nerede biter?
- Hangi kavram business domain içinde gerçekten anlam taşır?

Ekipler çoğu zaman problemi tam anlamadan teknoloji, framework, class tasarımı, database modeli veya mikroservis mimarisine atlar. Fakat problem yanlış kavramsallaştırılmışsa, seçilen teknoloji ne kadar doğru olursa olsun sistem zamanla bozulur.

Yanlış kavramsal modelin belirtileri şunlardır:

- İsimler yetersiz kalır.
- God object veya God service oluşur.
- Her yeni gereksinimde “bunu nereye koyacağız?” sorusu ortaya çıkar.
- Sürekli rename ve refactor ihtiyacı doğar.
- Teknik borç büyür.
- Domain kavramları ile kod kavramları birbirinden kopar.

Bu nedenle teknik borcun kök nedeni her zaman kötü kod değildir. Çoğu zaman asıl neden, başta yanlış kurulmuş kavramsal modeldir.

## 11.4 Kavramsallaştırma Örneği: Online Eğitim Platformu

Bir online eğitim platformunda ilk refleks şu teknik yapılara atlamak olabilir:

```
OrderService
PaymentService
RevenueService
PayoutService
Database tabloları
REST API tasarımı
Microservice mimarisi
```

Fakat doğru analiz için önce şu soru sorulmalıdır:

Bu sistemde gerçekten ne oluyor?

Bu soruya teknik terimlerle değil, domain diliyle cevap verilmelidir. Böyle bakıldığında sistemde şu kavramlar ortaya çıkar:

- erişim hakkı,
- satın alma,
- abonelik,

- hak ediş,
- gelir paylaşımı,
- ödeme,
- iade,
- politika.

Örneğin “eğitmen ne zaman para kazanır?” sorusu basit değildir. Eğitmen satın alma anında mı para kazanır, öğrenci kursu izledikçe mi, yoksa iade süresi dolunca mı?

Daha doğru cevap şudur:

Eğitmen, platformun tanımladığı hak ediş kuralına göre para kazanmış sayılır.

Bu durumda “hak ediş” merkezi bir domain kavramı haline gelir. Doğru kavramsal model şu tür cümlelerle ifade edilebilir:

Bir öğrenci bir erişim hakkı elde etti. Bu erişim belirli bir politikaya göre eğitmene hak ediş üretti. Bu hak ediş dönem sonunda bir ödemeye dönüştü.

Eğer sistem bu tür açık ve anlamlı cümlelerle anlatılabiliyorsa, model doğru yoldadır.

## 11.5 Basit Tasarımın Ana Fikri

Kent Beck’in basit tasarım yaklaşımı, gelecekte olabilecek her şeyi tahmin edip bugünden karmaşık yapılar kurmaya karşı çıkar. Gelecekteki değişiklikleri fazla tahmin etmeye çalışmak çoğu zaman daha iyi tasarım üretmez. Tam tersine, gereksiz abstraction, gereksiz class, gereksiz esneklik ve gereksiz karmaşıklık üretir.

Basit tasarımın temel fikri şudur:

Şu an testlerin ve gerçek ihtiyacın gerektirmediği hiçbir tasarım ögesini sisteme ekleme.

Yani iyi tasarım, her ihtimali bugünden kapsayan büyük ve soyut yapı değildir. İyi tasarım; bugünün ihtiyacını karşılayan, testlerle korunan, anlaşılır, tekrarsız ve gereksiz yük taşımayan tasarımdır.



## 11.6 Kent Beck'in Basit Tasarım İçin Dört Kuralı

### 1. Tüm Testler Çalışmalı

Önce sistem doğru çalışmalıdır. Testler, sistemin davranışını doğrulayan bir güvenlik ağıdır. Refactor veya yeni geliştirme yapıldığında sistemin hâlâ doğru çalıştığını testler üzerinden görebilmek gerekir.

Testler yoksa veya çalışmıyorsa, tasarımın ne kadar zarif olduğunun pratik değeri yoktur.

### 2. Kod Açık, Anlaşılır ve Tutarlı Olmalı

Kod bilgisayar tarafından çalıştırılır; fakat insanlar tarafından okunur. Bu nedenle kodun isimlendirmesi, akışı ve yapısı niyeti açıkça göstermelidir.

Burada *programming by intention* yaklaşımı önemlidir. Kod, implementasyon detaylarından önce niyeti göstermelidir. İsimler, değişkenler, fonksiyonlar ve sınıflar sistemde ne yapılmak istendiğini açıkça anlatmalıdır.

### 3. Tekrar Eden Yapı ve Davranış Olmamalı

Duplication, bakım maliyetini artırır. Aynı kavram, aynı algoritma, aynı veri veya aynı iş kuralı farklı yerlerde tekrar ederse sistem zamanla tutarsızlaşır.

Bu tekrar yalnızca kod tekrarı olmayabilir:

- kod tekrarı,
- model tekrarı,
- tip tekrarı,
- veri tekrarı,
- algoritma tekrarı,
- iş kuralı tekrarı.

Tekrar eden yapılar sistemden temizlenmelidir. Ancak bu işlem anlaşılabilirliği bozacak şekilde yapılmamalıdır.

### 4. Çözüm Mümkün Olan En Az Sınıf ve Metotla Kurulmalı

Her class ve method bir bakım yüküdür. İhtiyaç duyulmayan kod yazılmamalıdır. Eğer yazılmış ama artık ihtiyaç duyulmuyorsa silinmelidir.

Bu yaklaşım YAGNI prensibiyle ilişkilidir:

Şu an ihtiyacın yoksa yazma.

En iyi kod bazen hiç yazılmamış koddur. Çünkü her yeni kod, gelecekte okunması, anlaşılması, test edilmesi, debug edilmesi ve bakımının yapılması gereken bir yük oluşturur.

## 11.7 Basit Tasarım Kurallarında Öncelik Sırası

Bu dört kural aynı seviyede değildir. Aralarında öncelik sırası vardır:

1. Testler geçmeli.
2. Kod anlaşılır olmalı.
3. Duplication azaltılmalı.
4. Class ve metot sayısı minimum olmalı.

Örneğin duplication'ı azaltmak kodu daha anlaşılabilir hale getiriyorsa, anlaşılabilirlik daha önemlidir. Benzer şekilde kodun daha güzel görünmesi için testlerin bozulması kabul edilemez.

Önceki kural, sonraki kuraldan daha önceliklidir.

## 11.8 Unix Felsefesi ile Bağlantı

Basit tasarım anlayışı, Unix felsefesiyle de güçlü biçimde ilişkilidir. Unix felsefesinin temelinde büyük ve yekpare sistemler kurmak yerine, küçük, sade ve birbiriyle bağlanabilir parçalar üretmek vardır.

Unix felsefesinin bazı temel ilkeleri şunlardır:

- Küçük olan daha iyidir.
- Her program tek bir işi iyi yapmalıdır.
- Gereksiz çıktı üretmemek gerekir.
- Veri plain text tutulmalıdır.
- Kod ve veri taşınabilir olmalıdır.
- Programlar veri filtresi gibi tasarlanmalıdır.
- Captive UI'dan kaçınılmalıdır.
- Hiyerarşik düşünülmelidir.

### Portability Önemlidir

Kod ve veri farklı sistemlerde çalışabilir ve taşınabilir olmalıdır. Gereksiz erken optimizasyon yerine sürdürülebilirlik ve taşınabilirlik öncelenmelidir.

Bir sistemin uzun vadeli değeri, yalnızca bugünkü performansından değil, farklı ortamlara uyum sağlayabilmesinden de gelir.

## Captive UI'dan Kaçınmak Gerekir

Program kullanıcıyı kendi içine hapsetmemelidir. Başka araçlarla, komutlarla, scriptlerle ve sistemlerle birlikte çalışabilir olmalıdır.

Bu yaklaşım otomasyon ve entegrasyon açısından kritiktir. Bir program yalnızca insan etkileşimine göre tasarlanırsa, başka programlarla birlikte çalışabilme kapasitesi azalır.

## Hiyerarşik Düşünce Gerekir

Karmaşık sistemler rastgele değil, katmanlı ve hiyerarşik biçimde organize edilmelidir. Dosya sistemi, process yapısı, kernel-user space ayrımı veya modüler yazılım mimarisi gibi yapılar karmaşıklığı yönetilebilir hale getirir.

Karmaşıklık, büyük ve yekpare sistemler kurarak değil; küçük, iyi tanımlanmış parçaları belirli bir düzen içinde birleştirerek yönetilir.

## 11.9 Çeviklik ve Basit Tasarım

Basit tasarımın amacı yalnızca az kod yazmak değildir. Asıl amaç, proje ömrü boyunca hızlı ve sürekli çözüm üretebilme kabiliyetini korumaktır.

Çeviklik yalnızca sprint, backlog, retro veya estimation gibi süreçlerden doğmaz. Gerçek çeviklik; küçük adımlarla çalışan yazılım üretmek, gerçek kullanıcıdan geri bildirim almak ve sistemi bu geri bildirimlere göre evrilebilmektir.

Gerçek çeviklik, sistemi sürekli değişime açık tutabilme becerisidir.

## 11.10 Sonuç

Yazılım geliştirme yalnızca teknik bir faaliyet değil, aynı zamanda anlam inşa etme sürecidir. Kod, bu anlamın yalnızca bir yansımasıdır.

Kavramlar yanlışsa, en temiz kod bile sistemi kurtaramaz. Fakat kavramlar doğruysa; teknoloji değişse, mimari evrilse ve gereksinimler artsa bile sistem ayakta kalabilir.

Basit tasarım, doğru kavramsallaştırma ve Unix felsefesi aynı noktada birleşir:

Karmaşık sistemler, büyük ve karmaşık parçalarla değil; küçük, anlaşılır, test edilmiş, iyi adlandırılmış ve doğru biçimde birleştirilmiş parçalarla sürdürülebilir hale gelir.

## 12 Yazılımda Hikâyenin Önemi

### 12.1 Giriş

Yazılım geliştirmede çoğu zaman ilk refleks kod yazmak, teknoloji seçmek, veritabanı tasarlamak veya servisleri belirlemek olur. Ancak iyi bir yazılım sistemi kurmak için bundan önce yapılması gereken daha temel bir iş vardır: sistemin hikâyesini anlamak.

Burada hikâye ifadesi, pazarlama veya sunum amacıyla oluşturulan yüzeysel bir anlatı anlamına gelmez. Kastedilen şey, sistemin problem alanında gerçekte ne yaptığını insan diliyle açık ve tutarlı biçimde ifade edebilmektir.

İyi hikâye, sistemin ne yaptığını insan cümleleriyle anlatabilmektir.

### 12.2 Hikâye Neden Gereklidir?

Koddan, framework'ten, veritabanından, API tasarımından veya component yapısından önce şu soru sorulmalıdır:

Bu sistemde gerçekte ne oluyor?

Bu soru cevaplanmadan doğrudan teknik tasarıma geçildiğinde sistem genellikle teknik isimlerle dolmaya başlar:

```
UserService  
DataManager  
PaymentHelper  
OrderUtil  
ProcessHandler
```

Bu tür isimler çoğu zaman sistemin gerçek anlamını taşımaz. Çünkü domain hikâyesi netleşmediğinde, kavramlar da netleşmez. Kavramlar netleşmediğinde ise kod teknik parçaların toplamına dönüşür.

### 12.3 Hikâye ile Kavramların Ortaya Çıkması

Örneğin bir online eğitim platformunda teknik tasarıma hemen şu şekilde başlanabilir:

```
PaymentService yazacağız.  
Order tablosu olacak.  
Revenue hesaplayacağız.  
REST API oluşturacağız.
```

Fakat bu başlangıç teknik düzeydedir. Asıl problem alanını henüz açıklamaz. Daha doğru başlangıç, sistemdeki gerçek olayları anlatmaktır:

Bir öğrenci bir kursa erişim hakkı kazanır. Bu erişim satın alma, abonelik veya kampanya üzerinden oluşabilir. Erişim, belirli bir politikaya göre eğitime hak ediş üretir. Hak ediş dönem sonunda ödemeye dönüşür. İade olursa daha önce oluşan hak ediş geri alınabilir.

Bu anlatımda henüz teknoloji yoktur:

- API yoktur.
- Veritabanı yoktur.
- Mikroservis yoktur.
- React component yoktur.
- Class diagram yoktur.

Fakat sistemin temel kavramları görünmeye başlamıştır:

- öğrenci,
- kurs,
- erişim hakkı,
- satın alma,
- abonelik,
- kampanya,
- politika,
- hak ediş,
- iade,
- ödeme.

Bu nedenle hikâye, kavramları ortaya çıkaran ilk katmandır.

## 12.4 Hikâye Olmadan Kodun Durumu

Hikâye yoksa kod yalnızca teknik parçaların toplamı haline gelir. Bu durumda sistemde neyin neden var olduğu, hangi kavramın hangi davranışı temsil ettiği ve hangi sınırların korunması gerektiği belirsizleşir.

Hikâye yoksa kod teknik parçaların toplamı olur.

Böyle bir sistemde yeni bir gereksinim geldiğinde şu sorular sıkça ortaya çıkar:

- Bu davranışı hangi servise koymalıyız?
- Bu veri gerçekten hangi modele ait?
- Bu isim doğru mu?
- Bu kural ödeme ile mi, sipariş ile mi, hak ediş ile mi ilgili?
- İade olduğunda hangi kavram değişmeli?

Bu belirsizlik zamanla teknik borca dönüşür.

## 12.5 Hikâye Varsa Kodun Durumu

Hikâye varsa kod anlam taşıyan kavramların organizasyonu haline gelir. Sistem yalnızca çalışan koddan oluşmaz; aynı zamanda doğru isimlendirilmiş kavramlardan, net sorumluluklardan ve izlenebilir davranışlardan oluşur.

Hikâye varsa kod, anlam taşıyan kavramların organizasyonu olur.

Bu durumda kodun yapısı hikâyeden beslenir. Örneğin:

- `AccessRight` erişim hakkını temsil eder.
- `Entitlement` veya `Earning` hak ediş kavramını temsil eder.
- `RefundPolicy` iade davranışının kuralını temsil eder.
- `Payout` dönem sonunda gerçekleşen ödemeyi temsil eder.
- `SubscriptionAccessPolicy` aboneliğin hangi erişimleri sağladığını tanımlar.

Bu isimler yalnızca teknik isimler değildir. Her biri domain hikâyesindeki bir anlamı taşır.

## 12.6 Doğru Sıralama

İyi bir yazılım geliştirme sürecinde sıra şu şekilde olmalıdır:

1. Hikâye
2. Kavramlar
3. Sorumluluklar
4. İlişkiler
5. Davranışlar
6. Sınırlar
7. Test edilebilir kurallar
8. Kod

Bu sıra önemlidir. Çünkü kod, problem alanındaki anlamın makineye çevrilmiş halidir. Anlam net değilse, kod da zamanla bulanıklaşır.

## 12.7 Hikâneyi Çıkarmak İçin Sorulacak Sorular

Bir sistemi tasarlamadan önce şu sorular sorulmalıdır:

- Bu sistemde gerçekten ne oluyor?
- Kim, neyi, hangi hakla elde ediyor?
- Para, bilgi veya yetki ne zaman oluşuyor?
- Hangi olay hangi sonucu doğuruyor?
- Hangi kural hangi davranışı tetikliyor?
- Hangi kavram zaman içinde durum değiştiriyor?
- İptal, iade, hata veya başarısızlık durumunda ne geri alınır?
- Hangi kavram sistemin merkezinde yer alıyor?

Bu sorular, teknik tasarımdan önce domain hikâyesini görünür hale getirir.

## 12.8 Hikâyenin Yazılıma Katkısı

Hikâye netleştğinde yazılımın birçok alanı daha sağlıklı hale gelir:

- doğru isimlendirme kolaylaşır,
- domain model daha net kurulur,
- servis sınırları daha doğru çizilir,
- testler davranış odaklı yazılır,
- API contract daha anlamlı hale gelir,
- teknik borç azalır,
- yeni geliştiriciler sistemi daha hızlı anlar,
- AI agent projede daha az tahmin yapar ve daha çok iz sürebilir.

Bu nedenle hikâye yalnızca analiz aşamasına ait bir metin değildir. Hikâye, codebase'in anlam omurgasıdır.

Hikâye, codebase'in anlam omurgasıdır.

## 12.9 Sonuç

Yazılım geliştirme yalnızca teknik bir inşa süreci değildir. Aynı zamanda bir anlam inşa etme sürecidir. Kod, bu anlamın makineye çevrilmiş halidir.

Bu yüzden iyi bir yazılım sistemi kurmak için önce hikâye netleşmelidir. Hikâye netleşmeden kavramlar, sorumluluklar, ilişkiler ve sınırlar sağlıklı biçimde kurulamaz.

En kısa ifadeyle:

Önce hikâye kurulur, sonra kavramlar çıkarılır, sonra davranışlar tanımlanır, en son kod yazılır.